Mini project report on

DEFENCE ASSET READINESS & MAINTENANCE DEPENDENCY SYSTEM

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE23CS351A – DBMS Project

Submitted by:

Praneeth P Shetty	**PES2UG24CS902**
Raksha	**PES2UG24CS652**

under the guidance of

Dr. Shilpa S

Assistant Professor

PES University

AUG - DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

DEFENCE ASSET READINESS & MAINTENANCE DEPENDENCY SYSTEM

is a bonafide work carried out by

Praneeth P Shetty	**PES2UG24CS902**
Raksha	**PES2UG24CS652**

In partial fulfilment for the completion of fifth semester DBMS Project (UE23CS351A) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature
Dr. Shilpa S
Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **StudySync: A System for Fair Work Distribution and Efficient Group Meeting Coordination** has been carried out by us under the guidance of **Dr. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester AUG – DEC 2026.

Praneeth P Shetty	**PES2UG24CS902**	**<Signature>**
Raksha	**PES2UG24CS652**	**<Signature>**

ACKNOWLEDGEMENT

I would like to express my gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE23CS351A - DBMS Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

The Defence Asset Readiness and Maintenance Dependency System is a database-driven application designed to organize information about fictional defence-style assets, their organizational units, subsystems, components, component dependencies, spare parts, suppliers, failures, inspections and maintenance activities. The system provides a structured view of asset readiness and supports the analysis of maintenance history and technical dependencies.

The relational database is implemented in MySQL under the database name defence_asset_system. The verified schema contains 18 base tables connected through primary-key and foreign-key relationships. Relationship tables model component-to-component dependencies, component-to-spare mappings and supplier-to-spare mappings. The database also contains an asset readiness view, a trigger that updates an asset's status when a repair event is inserted, a stored function for counting failures associated with an asset, and a stored procedure for updating asset status.

A web application built with Next.js and TypeScript provides modules for dashboard summaries, asset records, work orders, component dependencies, spare-part inventory and reports. The application accesses MySQL through server-side API route handlers using parameterized queries and input validation. Redis is included as an optional key-value cache for the dashboard summary; MySQL remains the authoritative data store. The report describes the problem, requirements, ER model, relational mapping, SQL implementation, application architecture and verification boundaries.

The project uses synthetic academic data and does not represent an official military readiness standard or contain classified operational information.

Keywords: DBMS, MySQL, asset readiness, maintenance management, component dependency, triggers, stored procedures, Next.js, Redis.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	9
2.	PROBLEM DEFINITION	11
3.	ER MODEL	13
4.	ER TO RELATIONAL MAPPING	16
5.	DDL STATEMENTS	19
6.	DML STATEMENTS	35
7.	QUERIES (SIMPLE QUERY AND UPDATE AND DELETE OPERATION, CORRELATED QUERY AND NESTED QUERY)	46
8.	STORED PROCEDURE, FUNCTIONS AND TRIGGERS	62
9.	FRONT END DEVELOPMENT	75
REFERENCES/BIBLIOGRAPHY		88
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		89

LIST OF TABLES

Table No.	Title	Page No
1.1	Technology stack	10
2.1	Functional requirements	11
3.1	Entities and key attributes	13
3.2	Relationship cardinalities	14
9.1	Main API endpoint groups	81
A.1	Test Evidence Tables	90

LIST OF FIGURES

Figure No.	Title	Page No
1.1	High-level system architecture	10
3.1	Entity-relationship diagram	15
4.1	Relational schema diagram	18
5.1	MySQL database and table creation evidence	19
6.1	Inserted data evidence	35
7.1	Representative SQL query outputs	52
8.1	Stored function, procedure and trigger evidence	68
9	User Interface	81
9.10	Redis health status	87

1. INTRODUCTION

1.1 Background

Organizations that operate complex equipment need to maintain accurate records of assets, their components, failures, inspections, maintenance work orders, spare-part availability and suppliers. These records are related: a failure is associated with an asset, a work order belongs to an asset, a maintenance event belongs to a work order, and components may depend on other components or require specific spare parts.

When this information is maintained in disconnected files or unstructured records, it becomes difficult to identify relationships, review maintenance history, locate low-stock spare parts and understand which assets may be affected by a component dependency. A relational database provides a consistent structure for storing these entities and connecting them through keys.

1.2 About Project

The Defence Asset Readiness and Maintenance Dependency System is an academic decision-support application built around a MySQL relational database. It organizes fictional defence-style asset information and provides application modules for assets, work orders, component dependencies, inventory and reports. The system is intended to demonstrate database design and SQL concepts rather than to act as an authoritative operational readiness platform.

1.3 Objectives

- Design a relational database for assets, units, services, components, spare parts, suppliers, failures and maintenance records.
- Represent one-to-many and many-to-many relationships using primary keys and foreign keys.
- Support queries for asset status, failure history, maintenance cost, downtime and spare-part information.
- Demonstrate SQL features including joins, grouping, aggregate functions, set operations, nested and correlated subqueries, views, indexes, triggers, stored routines and transactions.
- Provide a web interface for viewing and managing selected asset, work-order, dependency and inventory information.
- Apply server-side validation, parameterized SQL and role-based access checks in the application.

1.4 Technology Stack

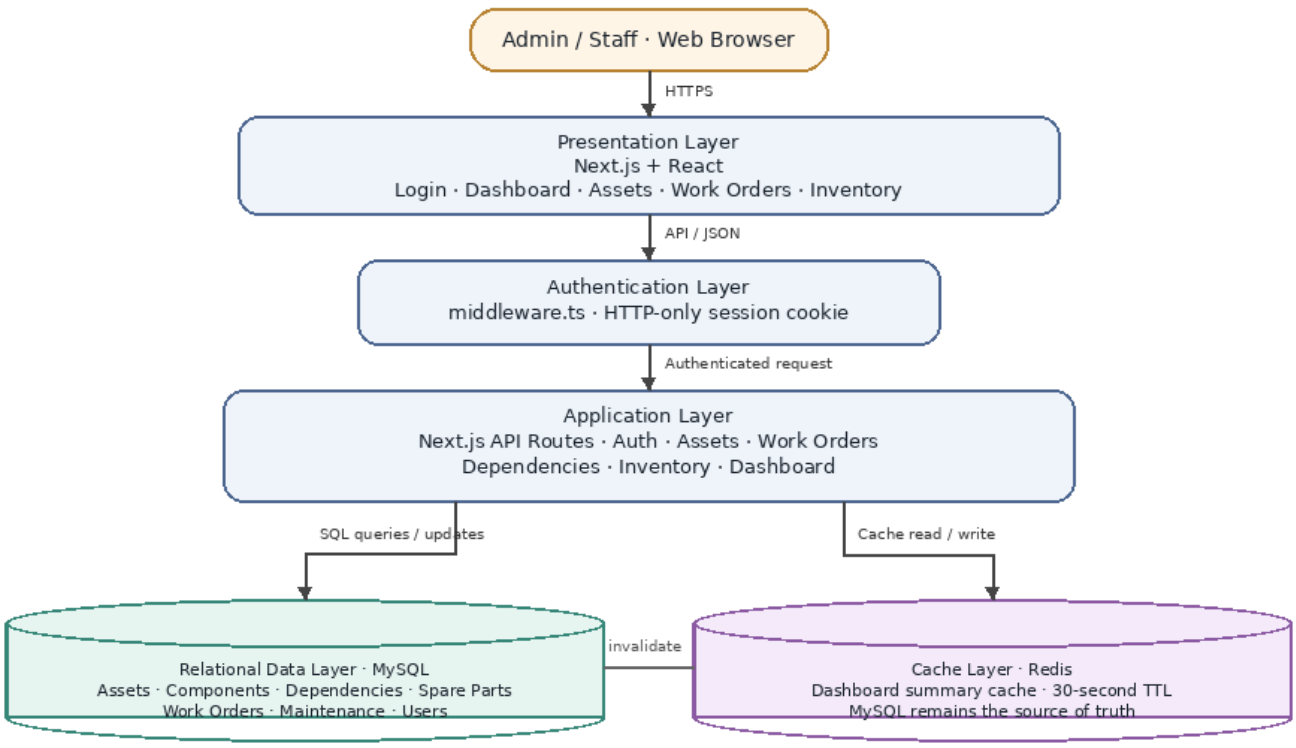
Table 1.1: Technology stack used in StudySync

Layer	Technology / purpose
Relational database	MySQL 8.0; authoritative structured data store
Frontend	Next.js App Router, React and TypeScript
Backend	Next.js Route Handlers / server-side API
Database connectivity	mysql2 connection pool and parameterized queries
Validation	Zod request validation
Authentication	Signed HTTP-only session cookie; PBKDF2-SHA256 password verification
Optional cache	Redis key-value cache for dashboard summary (30-second TTL)
Development tools	MySQL Workbench/CLI, Node.js, npm, Git-compatible source files

1.5 System Architecture

The browser communicates with the Next.js application. Server-side route handlers validate requests, apply authentication and authorization checks, and execute parameterized SQL against MySQL. MySQL stores the authoritative records. The dashboard may use Redis to cache a computed summary for a short period; if Redis is unavailable or the key expires, the application falls back to MySQL. Redis is therefore an optional cache and not a replacement for the relational database.

Figure 1.1 — High-level system architecture



2. PROBLEM DEFINITION

2.1 Problem Statement

To design and implement a database-driven system that stores and relates fictional asset, component, dependency, spare-part, supplier, failure, inspection and maintenance information, and provides queries and application views that help users review asset status, maintenance activity and inventory information while preserving referential integrity.

2.2 Scope

The project covers relational data modelling, SQL implementation, asset and maintenance records, component dependency mappings, spare-part inventory, supplier mappings, failure and inspection history, selected analytical queries and a web interface. Redis is used only as an optional cache for dashboard summaries.

The project does not claim to use classified defence data, implement an official military readiness calculation, or provide a certified operational decision system. The database and domain records are synthetic academic data.

2.3 Functional Requirements

Table 2.1: Functional requirements

ID	Requirement
FR1	Store countries, services and units and relate them through foreign keys.
FR2	Store asset types and assets and associate each asset with a unit and asset type.
FR3	Represent assets, subsystems and components in a hierarchy.
FR4	Represent component dependencies and component-to-spare-part mappings.
FR5	Maintain supplier information and supplier-to-spare-part mappings.
FR6	Record asset failures and inspection information.
FR7	Create maintenance work orders for assets and record maintenance events against work orders.
FR8	Provide SQL queries and a view for asset readiness and maintenance analysis.
FR9	Provide application modules for assets, work orders, dependencies, inventory and reports.
FR10	Protect application routes and validate user input on the server.

2.4 Non-Functional Requirements

- Data integrity through primary keys, foreign keys and the constraints present in the schema.
- Security through server-side authentication, authorization checks, protected environment variables and parameterized SQL.
- Maintainability through separated UI, API route handlers and database utility modules.
- Availability of core database-backed operations when the optional Redis cache is unavailable.
- Clarity of reporting through readable result sets, summaries and maintenance information.

2.5 Business Rules and Assumptions

- Each asset belongs to one unit and one asset type.
- Each subsystem belongs to an asset; each component belongs to a subsystem.
- Component dependencies are represented by pairs of component identifiers.
- A component can map to multiple spare parts, and a spare part can support multiple components.
- A maintenance work order belongs to an asset; maintenance events belong to a work order and identify the user who performed the work.
- The repair trigger sets the associated asset status to Ready when a Repair maintenance event is inserted. This is the implemented demonstration rule, not a general readiness standard.
- The application assumes Admin and Analyst role values for its demonstration authorization model.

3. ER MODEL

The ER model is derived from the verified MySQL schema export Dump20261101.sql. It contains 18 base relations. Entity tables represent organizational, asset, inventory, user and maintenance information. Three relationship tables use composite primary keys to represent component dependencies, component-spare mappings and supplier-spare mappings.

3.1 Entities and Key Attributes

Table 3.1: Entities and Key Attributes

Entity	Primary key	Purpose
COUNTRY	CountryID	Stores country and region information.
SERVICE	ServiceID	Stores service information and references a country.
UNIT	UnitID	Stores units and references a service.
ASSET_TYPE	AssetTypeID	Classifies assets by type/category.
ASSET	AssetID	Stores asset identity, model, status, criticality, unit and type.
SUBSYSTEM	SubsystemID	Stores subsystems belonging to an asset.
COMPONENT	ComponentID	Stores components belonging to a subsystem.
COMPONENT_DEPENDENCY	(ComponentID, DependsOnComponentID)	Maps a component to another component it depends on.
SPARE_PART	SparePartID	Stores spare part stock, reorder level and cost.
COMPONENT_SPARE	(ComponentID, SparePartID)	Maps components to required spare parts.
SUPPLIER	SupplierID	Stores supplier contact information.
SUPPLIER_SPARE	(SupplierID, SparePartID)	Maps suppliers to spare parts and supply details.
FAILURE_EVENT	FailureID	Stores failure type, severity, time, description and asset.
INSPECTION	InspectionID	Stores inspection results associated with a failure and inspector.
MAINTENANCE_WORK_ORDER	WorkOrderID	Stores maintenance work orders associated with assets.
MAINTENANCE_EVENT	MaintenanceID	Stores maintenance activity, performer, time, type, action, downtime and cost.
USER	UserID	Stores application/user identity, role and contact fields.

3.2 Relationships and Cardinality

Table 3.2: Relationship types

Relationship	Cardinality	Explanation
COUNTRY–SERVICE	1:N	A country can be associated with multiple services; each service references one country.
SERVICE–UNIT	1:N	A service can contain multiple units; each unit references one service.
UNIT–ASSET	1:N	A unit can have multiple assets; each asset references one unit.
ASSET_TYPE–ASSET	1:N	An asset type can classify multiple assets.
ASSET–SUBSYSTEM	1:N	An asset can contain multiple subsystems.
SUBSYSTEM–COMPONENT	1:N	A subsystem can contain multiple components.
COMPONENT–COMPONENT	M:N	Self-referential dependency represented by COMPONENT_DEPENDENCY.
COMPONENT–SPARE_PART	M:N	Mapped through COMPONENT_SPARE, with quantity/criticality attributes.
SUPPLIER–SPARE_PART	M:N	Mapped through SUPPLIER_SPARE, with lead time/price/preference attributes.
ASSET–FAILURE_EVENT	1:N	An asset can have multiple failure records.
FAILURE_EVENT–INSPECTION	1:N	A failure can have multiple inspection records.
ASSET–MAINTENANCE_WORK_ORDER	1:N	An asset can have multiple work orders.
MAINTENANCE_WORK_ORDER–MAINTENANCE_EVENT	1:N	A work order can have multiple maintenance events.
USER–INSPECTION / MAINTENANCE_EVENT	1:N	A user can perform multiple inspections or maintenance events.

3.3 ER Diagram

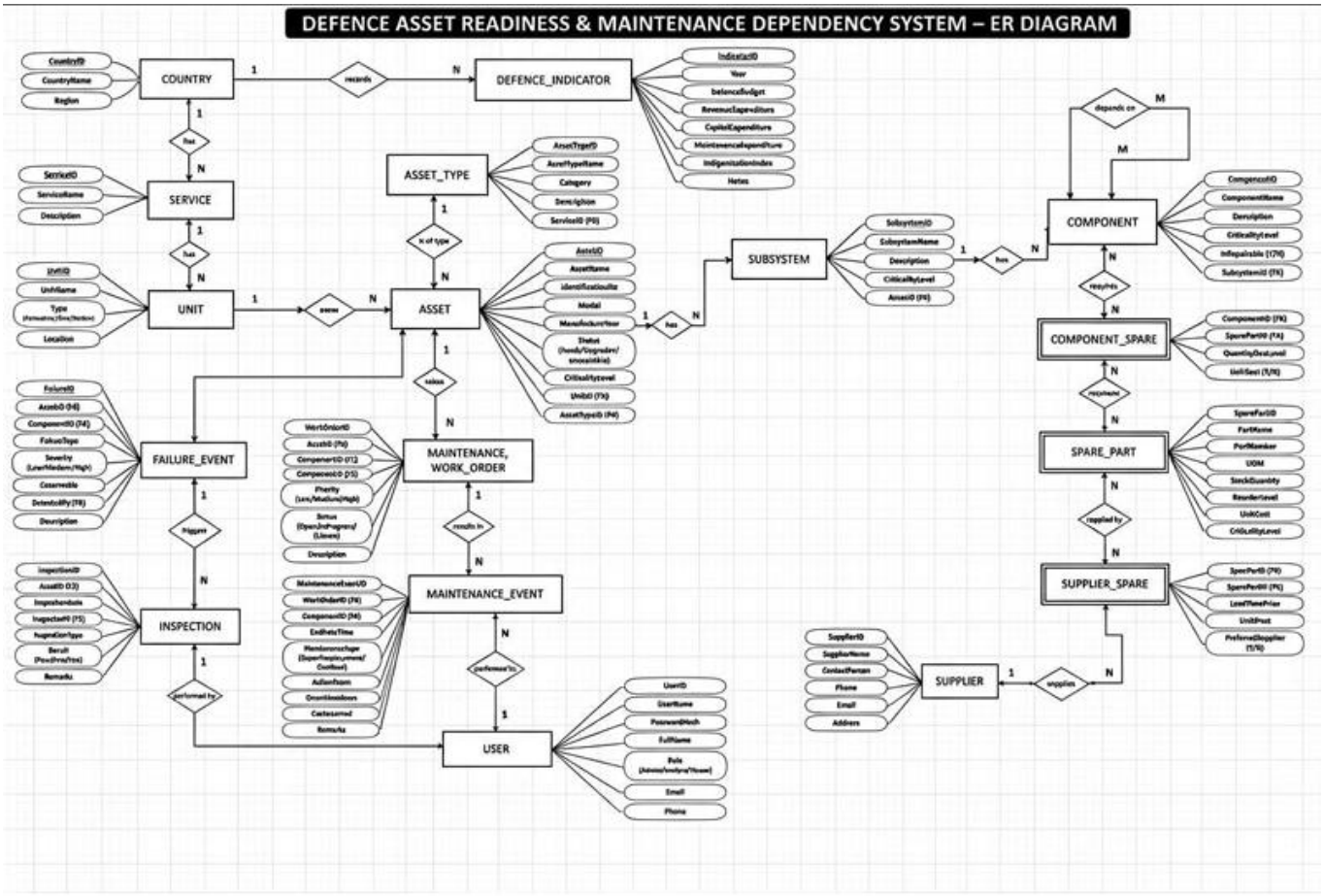


Figure 3.1 — ER diagram

4 ER TO RELATIONAL MAPPING

4.1 Mapping of Strong Entities

Each regular entity type is mapped to a relation containing its simple attributes. The entity's identifier becomes the primary key. For example, ASSET becomes ASSET(AssetID, AssetName, IdentificationNo, Model, MnfYear, Status, CriticalityLevel, UnitID, AssetTypeID).

4.2 Mapping of 1:N Relationships

For each one-to-many relationship, the primary key of the one-side is stored as a foreign key in the many-side relation. Examples include SERVICE.CountryID referencing COUNTRY, UNIT.ServiceID referencing SERVICE, ASSET.UnitID referencing UNIT, ASSET.AssetTypeID referencing ASSET_TYPE, SUBSYSTEM.AssetID referencing ASSET and COMPONENT.SubsystemID referencing SUBSYSTEM.

4.3 Mapping of M:N Relationships

Many-to-many relationships are represented using separate relationship relations. COMPONENT_DEPENDENCY uses the composite key (ComponentID, DependsOnComponentID). COMPONENT_SPARE uses (ComponentID, SparePartID) and stores QuantityRequired and IsCritical. SUPPLIER_SPARE uses (SupplierID, SparePartID) and stores LeadTimeDay, UnitPrice and PreferredSupplier.

4.4 Resulting Relational Schema

- COUNTRY(CountryID PK, CountryName, Region)
- SERVICE(ServiceID PK, ServiceName, Description, CountryID FK)
- UNIT(UnitID PK, UnitName, Type, Location, ServiceID FK)
- ASSET_TYPE(AssetTypeID PK, AssetTypeName, Category, Description)
- ASSET(AssetID PK, AssetName, IdentificationNo, Model, MnfYear, Status, CriticalityLevel, UnitID FK, AssetTypeID FK)
- SUBSYSTEM(SubsystemID PK, SubsystemName, Description, CriticalityLevel, AssetID FK)
- COMPONENT(ComponentID PK, ComponentName, Description, CriticalityLevel, IsRepairable, SubsystemID FK)
- COMPONENT_DEPENDENCY(ComponentID PK/FK, DependsOnComponentID PK/FK)
- SPARE_PART(SparePartID PK, PartName, PartNumber, UOM, StockQuantity, ReorderLevel, UnitCost, CriticalityLevel)
- COMPONENT_SPARE(ComponentID PK/FK, SparePartID PK/FK, QuantityRequired, IsCritical)
- SUPPLIER(SupplierID PK, SupplierName, ContactPerson, Phone, Email, Address)
- SUPPLIER_SPARE(SupplierID PK/FK, SparePartID PK/FK, LeadTimeDay, UnitPrice, PreferredSupplier)
- FAILURE_EVENT(FailureID PK, FailureType, Severity, OccurredAt, Description, AssetID FK)
- INSPECTION(InspectionID PK, FailureID FK, InspectorID FK, InspectionType, InspectionDate, Result, Remarks)
- MAINTENANCE_WORK_ORDER(WorkOrderID PK, AssetID FK, Description, OrderDate, Priority, Status)
- MAINTENANCE_EVENT(MaintenanceID PK, WorkOrderID FK, PerformedBy FK, Remarks, StartDateTime, EndDateTime, MaintenanceType, ActionTaken, DowntimeHours, CostIncurred)
- USER(UserID PK, UserName, PasswordHash, FullName, Role, Email, Phone)

4.5 Normalization

The exported schema stores scalar values in individual columns and does not place repeating groups inside a row, supporting First Normal Form (1NF). Entity relations use single-column primary keys. Relationship attributes in the three composite-key relations depend on the complete composite key, with no partial dependency visible in the documented design; this supports Second Normal Form (2NF). Under the functional dependencies guaranteed by the declared keys, non-key attributes depend on their relation's key, and referenced entities are separated into their own relations, supporting Third Normal Form (3NF).

The documented key-based dependencies also satisfy BCNF under the assumptions stated in docs/normalization_and_design.md. This conclusion is limited to dependencies enforced or implied by the exported keys; fields such as email, part number and identification number are not declared as alternate unique keys in the export.



Fig 4.1: Relational schema diagram

5 DDL STATEMENTS

The database definition is implemented in MySQL. The supplied export is Dump20261101.sql and identifies the database as defence_asset_system. It contains CREATE TABLE definitions, primary keys, foreign keys, indexes, a view, a trigger, a stored function and a stored procedure.

5.1 Table COUNTRY:

Code:

```
CREATE TABLE COUNTRY (
    CountryID INT PRIMARY KEY,
    CountryName VARCHAR(100) NOT NULL,
    Region VARCHAR(100)
);
```

Screenshot:

```
mysql> CREATE TABLE COUNTRY (
    ->     CountryID INT PRIMARY KEY,
    ->     CountryName VARCHAR(100) NOT NULL,
    ->     Region VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> Desc country;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| CountryID   | int          | NO   | PRI | NULL    |       |
| CountryName | varchar(100) | NO   |     | NULL    |       |
| Region      | varchar(100) | YES  |     | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
3 rows in set (0.01 sec)
```

5.2 Table USER:

Code:

```
CREATE TABLE `USER` (
    UserID INT PRIMARY KEY,
    UserName VARCHAR(50) NOT NULL,
    PasswordHash VARCHAR(255) NOT NULL,
    FullName VARCHAR(100),
    Role VARCHAR(30),
    Email VARCHAR(100),
    Phone VARCHAR(20)
);
```

Screenshot:

```
mysql> CREATE TABLE `USER` (
    ->     UserID INT PRIMARY KEY,
    ->     UserName VARCHAR(50) NOT NULL,
    ->     PasswordHash VARCHAR(255) NOT NULL,
    ->     FullName VARCHAR(100),
    ->     Role VARCHAR(30),
    ->     Email VARCHAR(100),
    ->     Phone VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> DESCRIBE `USER`;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| UserID       | int          | NO   | PRI | NULL    |       |
| UserName     | varchar(50)  | NO   |     | NULL    |       |
| PasswordHash | varchar(255) | NO   |     | NULL    |       |
| FullName     | varchar(100) | YES  |     | NULL    |       |
| Role         | varchar(30)  | YES  |     | NULL    |       |
| Email        | varchar(100) | YES  |     | NULL    |       |
| Phone        | varchar(20)  | YES  |     | NULL    |       |
+--------------+--------------+------+-----+---------+-------+
7 rows in set (0.01 sec)
```

5.3 Table SERVICE:

Code:

```sql
CREATE TABLE SERVICE (
    ServiceID INT PRIMARY KEY,
    ServiceName VARCHAR(100) NOT NULL,
    Description VARCHAR(255),
    CountryID INT NOT NULL,

    FOREIGN KEY (CountryID)
        REFERENCES COUNTRY(CountryID)
);
```

Screenshot:

```
mysql> CREATE TABLE SERVICE (
    ->     ServiceID INT PRIMARY KEY,
    ->     ServiceName VARCHAR(100) NOT NULL,
    ->     Description VARCHAR(255),
    ->     CountryID INT NOT NULL,
    ->     FOREIGN KEY (CountryID) REFERENCES COUNTRY(Country
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> DESCRIBE SERVICE;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| ServiceID   | int          | NO   | PRI | NULL    |       |
| ServiceName | varchar(100) | NO   |     | NULL    |       |
| Description | varchar(255) | YES  |     | NULL    |       |
| CountryID   | int          | NO   | MUL | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
4 rows in set (0.01 sec)
```

5.4 Table DEFENCE_INDICATOR:

Code:

```
CREATE TABLE DEFENCE_INDICATOR (
    IndicatorID INT PRIMARY KEY,
    CountryID INT NOT NULL,
    Year INT NOT NULL,
    DefenceBudget DECIMAL(15,2),
    RevenueExpenditure DECIMAL(15,2),
    CapitalExp DECIMAL(15,2),
    MaintenanceExp DECIMAL(15,2),
    IndigenisationIndex DECIMAL(5,2),
    Notes VARCHAR(500),

    FOREIGN KEY (CountryID)
        REFERENCES COUNTRY(CountryID)
);
 --
```

Screenshot:

```
mysql> CREATE TABLE DEFENCE_INDICATOR (
    ->     IndicatorID INT PRIMARY KEY,
    ->     CountryID INT NOT NULL,
    ->     Year INT NOT NULL,
    ->     DefenceBudget DECIMAL(15,2),
    ->     RevenueExpenditure DECIMAL(15,2),
    ->     CapitalExp DECIMAL(15,2),
    ->     MaintenanceExp DECIMAL(15,2),
    ->     IndigenisationIndex DECIMAL(5,2),
    ->     Notes VARCHAR(500),
    ->     FOREIGN KEY (CountryID) REFERENCES COUNTRY(CountryID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DESCRIBE DEFENCE_INDICATOR;
+---------------------+---------------+------+-----+---------+-------+
| Field               | Type          | Null | Key | Default | Extra |
+---------------------+---------------+------+-----+---------+-------+
| IndicatorID         | int           | NO   | PRI | NULL    |       |
| CountryID           | int           | NO   | MUL | NULL    |       |
| Year                | int           | NO   |     | NULL    |       |
| DefenceBudget       | decimal(15,2) | YES  |     | NULL    |       |
| RevenueExpenditure  | decimal(15,2) | YES  |     | NULL    |       |
| CapitalExp          | decimal(15,2) | YES  |     | NULL    |       |
| MaintenanceExp      | decimal(15,2) | YES  |     | NULL    |       |
| IndigenisationIndex | decimal(5,2)  | YES  |     | NULL    |       |
| Notes               | varchar(500)  | YES  |     | NULL    |       |
+---------------------+---------------+------+-----+---------+-------+
9 rows in set (0.00 sec)
```

5.5 Table UNIT:
Code:

```sql
CREATE TABLE UNIT (
    UnitID INT PRIMARY KEY,
    UnitName VARCHAR(100) NOT NULL,
    Type VARCHAR(50),
    Location VARCHAR(100),
    ServiceID INT NOT NULL,

    FOREIGN KEY (ServiceID)
        REFERENCES SERVICE(ServiceID)
);
```

Screenshot:

```
mysql> CREATE TABLE UNIT (
    ->     UnitID INT PRIMARY KEY,
    ->     UnitName VARCHAR(100) NOT NULL,
    ->     Type VARCHAR(50),
    ->     Location VARCHAR(100),
    ->     ServiceID INT NOT NULL,
    ->     FOREIGN KEY (ServiceID) REFERENCES SERVICE(ServiceID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DESCRIBE UNIT;
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| UnitID    | int          | NO   | PRI | NULL    |       |
| UnitName  | varchar(100) | NO   |     | NULL    |       |
| Type      | varchar(50)  | YES  |     | NULL    |       |
| Location  | varchar(100) | YES  |     | NULL    |       |
| ServiceID | int          | NO   | MUL | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
5 rows in set (0.01 sec)
```

5.6 Table ASSET_TYPE:

Code:

```sql
CREATE TABLE ASSET_TYPE (
    AssetTypeID INT PRIMARY KEY,
    AssetTypeName VARCHAR(100) NOT NULL,
    Category VARCHAR(100),
    Description VARCHAR(255)
);
```

Screenshot:

```
mysql> CREATE TABLE ASSET_TYPE (
    ->     AssetTypeID INT PRIMARY KEY,
    ->     AssetTypeName VARCHAR(100) NOT NULL,
    ->     Category VARCHAR(100),
    ->     Description VARCHAR(255)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> DESCRIBE ASSET_TYPE;
+---------------+--------------+------+-----+---------+-------+
| Field         | Type         | Null | Key | Default | Extra |
+---------------+--------------+------+-----+---------+-------+
| AssetTypeID   | int          | NO   | PRI | NULL    |       |
| AssetTypeName | varchar(100) | NO   |     | NULL    |       |
| Category      | varchar(100) | YES  |     | NULL    |       |
| Description   | varchar(255) | YES  |     | NULL    |       |
+---------------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

5.7 Table ASSET:

Code:

```sql
CREATE TABLE ASSET (
    AssetID INT PRIMARY KEY,
    AssetName VARCHAR(100) NOT NULL,
    IdentificationNo VARCHAR(100),
    Model VARCHAR(100),
    MnfYear INT,
    Status VARCHAR(30),
    CriticalityLevel VARCHAR(20),
    UnitID INT NOT NULL,
    AssetTypeID INT NOT NULL,

    FOREIGN KEY (UnitID)
        REFERENCES UNIT(UnitID),

    FOREIGN KEY (AssetTypeID)
        REFERENCES ASSET_TYPE(AssetTypeID)
);
```

Screenshot:

```
mysql> CREATE TABLE ASSET (
    ->     AssetID INT PRIMARY KEY,
    ->     AssetName VARCHAR(100) NOT NULL,
    ->     IdentificationNo VARCHAR(100),
    ->     Model VARCHAR(100),
    ->     MnfYear INT,
    ->     Status VARCHAR(30),
    ->     CriticalityLevel VARCHAR(20),
    ->     UnitID INT NOT NULL,
    ->     AssetTypeID INT NOT NULL,
    ->     FOREIGN KEY (UnitID) REFERENCES UNIT(UnitID),
    ->     FOREIGN KEY (AssetTypeID) REFERENCES ASSET_TYPE(AssetTypeI
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> DESCRIBE ASSET;
+------------------+-------------+------+-----+---------+-------+
| Field            | Type        | Null | Key | Default | Extra |
+------------------+-------------+------+-----+---------+-------+
| AssetID          | int         | NO   | PRI | NULL    |       |
| AssetName        | varchar(100)| NO   |     | NULL    |       |
| IdentificationNo | varchar(100)| YES  |     | NULL    |       |
| Model            | varchar(100)| YES  |     | NULL    |       |
| MnfYear          | int         | YES  |     | NULL    |       |
| Status           | varchar(30) | YES  |     | NULL    |       |
| CriticalityLevel | varchar(20) | YES  |     | NULL    |       |
| UnitID           | int         | NO   | MUL | NULL    |       |
| AssetTypeID      | int         | NO   | MUL | NULL    |       |
+------------------+-------------+------+-----+---------+-------+
9 rows in set (0.01 sec)
```

5.8 Table FAILURE_EVENT:

Code:

```sql
CREATE TABLE FAILURE_EVENT (
    FailureID INT PRIMARY KEY,
    FailureType VARCHAR(100),
    Severity VARCHAR(20),
    OccurredAt DATETIME,
    Description VARCHAR(500),
    AssetID INT NOT NULL,

    FOREIGN KEY (AssetID)
        REFERENCES ASSET(AssetID)
);
```

Screenshot:

```
mysql> CREATE TABLE FAILURE_EVENT (
    ->     FailureID INT PRIMARY KEY,
    ->     FailureType VARCHAR(100),
    ->     Severity VARCHAR(20),
    ->     OccurredAt DATETIME,
    ->     Description VARCHAR(500),
    ->     AssetID INT NOT NULL,
    ->     FOREIGN KEY (AssetID) REFERENCES ASSET(AssetID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DESCRIBE FAILURE_EVENT;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| FailureID   | int          | NO   | PRI | NULL    |       |
| FailureType | varchar(100) | YES  |     | NULL    |       |
| Severity    | varchar(20)  | YES  |     | NULL    |       |
| OccurredAt  | datetime     | YES  |     | NULL    |       |
| Description | varchar(500) | YES  |     | NULL    |       |
| AssetID     | int          | NO   | MUL | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
6 rows in set (0.01 sec)
```

5.9 Table INSPECTION:

Code:

```sql
CREATE TABLE INSPECTION (
    InspectionID INT PRIMARY KEY,
    FailureID INT NOT NULL,
    InspectorID INT NOT NULL,
    InspectionType VARCHAR(100),
    InspectionDate DATE,
    Result VARCHAR(50),
    Remarks VARCHAR(500),

    FOREIGN KEY (FailureID)
        REFERENCES FAILURE_EVENT(FailureID),

    FOREIGN KEY (InspectorID)
        REFERENCES `USER`(UserID)
);
```

Screenshot:

```
mysql> CREATE TABLE INSPECTION (
    ->     InspectionID INT PRIMARY KEY,
    ->     FailureID INT NOT NULL,
    ->     InspectorID INT NOT NULL,
    ->     InspectionType VARCHAR(100),
    ->     InspectionDate DATE,
    ->     Result VARCHAR(50),
    ->     Remarks VARCHAR(500),
    ->     FOREIGN KEY (FailureID) REFERENCES FAILURE_EVENT(FailureID),
    ->     FOREIGN KEY (InspectorID) REFERENCES `USER`(UserID)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> DESCRIBE INSPECTION;
+----------------+--------------+------+-----+---------+-------+
| Field          | Type         | Null | Key | Default | Extra |
+----------------+--------------+------+-----+---------+-------+
| InspectionID   | int          | NO   | PRI | NULL    |       |
| FailureID      | int          | NO   | MUL | NULL    |       |
| InspectorID    | int          | NO   | MUL | NULL    |       |
| InspectionType | varchar(100) | YES  |     | NULL    |       |
| InspectionDate | date         | YES  |     | NULL    |       |
| Result         | varchar(50)  | YES  |     | NULL    |       |
| Remarks        | varchar(500) | YES  |     | NULL    |       |
+----------------+--------------+------+-----+---------+-------+
7 rows in set (0.00 sec)
```

5.10 Table MAINTENANCE_WORK_ORDER:

Code:

```sql
CREATE TABLE MAINTENANCE_WORK_ORDER (
    WorkOrderID INT PRIMARY KEY,
    AssetID INT NOT NULL,
    Description VARCHAR(500),
    OrderDate DATE,
    Priority VARCHAR(20),
    Status VARCHAR(30),

    FOREIGN KEY (AssetID)
        REFERENCES ASSET(AssetID)
);
```

Screenshot:

```
mysql> CREATE TABLE MAINTENANCE_WORK_ORDER (
    ->     WorkOrderID INT PRIMARY KEY,
    ->     AssetID INT NOT NULL,
    ->     Description VARCHAR(500),
    ->     OrderDate DATE,
    ->     Priority VARCHAR(20),
    ->     Status VARCHAR(30),
    ->     FOREIGN KEY (AssetID) REFERENCES ASSET(AssetID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DESCRIBE MAINTENANCE_WORK_ORDER;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| WorkOrderID | int          | NO   | PRI | NULL    |       |
| AssetID     | int          | NO   | MUL | NULL    |       |
| Description | varchar(500) | YES  |     | NULL    |       |
| OrderDate   | date         | YES  |     | NULL    |       |
| Priority    | varchar(20)  | YES  |     | NULL    |       |
| Status      | varchar(30)  | YES  |     | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
6 rows in set (0.00 sec)
```

5.11 Table MAINTENANCE_EVENT:

Code:

```sql
CREATE TABLE MAINTENANCE_EVENT (
    MaintenanceID INT PRIMARY KEY,
    WorkOrderID INT NOT NULL,
    Remarks VARCHAR(500),
    PerformedBy INT NOT NULL,
    StartDateTime DATETIME,
    EndDateTime DATETIME,
    MaintenanceType VARCHAR(50),
    ActionTaken VARCHAR(500),
    DowntimeHours DECIMAL(10,2),
    CostIncurred DECIMAL(15,2),

    FOREIGN KEY (WorkOrderID)
        REFERENCES MAINTENANCE_WORK_ORDER(WorkOrderID),

    FOREIGN KEY (PerformedBy)
        REFERENCES `USER`(UserID)
);
```

Screenshot:

```
mysql> CREATE TABLE MAINTENANCE_EVENT (
    ->     MaintenanceID INT PRIMARY KEY,
    ->     WorkOrderID INT NOT NULL,
    ->     Remarks VARCHAR(500),
    ->     PerformedBy INT NOT NULL,
    ->     StartDateTime DATETIME,
    ->     EndDateTime DATETIME,
    ->     MaintenanceType VARCHAR(50),
    ->     ActionTaken VARCHAR(500),
    ->     DowntimeHours DECIMAL(10,2),
    ->     CostIncurred DECIMAL(15,2),
    ->     FOREIGN KEY (WorkOrderID) REFERENCES MAINTENANCE_WORK_ORDER(W
orkOrderID),
    ->     FOREIGN KEY (PerformedBy) REFERENCES `USER`(UserID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DESCRIBE MAINTENANCE_EVENT;
+-----------------+---------------+------+-----+---------+-------+
| Field           | Type          | Null | Key | Default | Extra |
+-----------------+---------------+------+-----+---------+-------+
| MaintenanceID   | int           | NO   | PRI | NULL    |       |
| WorkOrderID     | int           | NO   | MUL | NULL    |       |
| Remarks         | varchar(500)  | YES  |     | NULL    |       |
| PerformedBy     | int           | NO   | MUL | NULL    |       |
| StartDateTime   | datetime      | YES  |     | NULL    |       |
| EndDateTime     | datetime      | YES  |     | NULL    |       |
| MaintenanceType | varchar(50)   | YES  |     | NULL    |       |
| ActionTaken     | varchar(500)  | YES  |     | NULL    |       |
| DowntimeHours   | decimal(10,2) | YES  |     | NULL    |       |
| CostIncurred    | decimal(15,2) | YES  |     | NULL    |       |
+-----------------+---------------+------+-----+---------+-------+
10 rows in set (0.00 sec)
```

5.12 Table SUBSYSTEM:

Code:

```sql
CREATE TABLE SUBSYSTEM (
    SubsystemID INT PRIMARY KEY,
    SubsystemName VARCHAR(100) NOT NULL,
    Description VARCHAR(255),
    CriticalityLevel VARCHAR(20),
    AssetID INT NOT NULL,

    FOREIGN KEY (AssetID)
        REFERENCES ASSET(AssetID)
);
```

Screenshot:

```
mysql> CREATE TABLE SUBSYSTEM (
    ->     SubsystemID INT PRIMARY KEY,
    ->     SubsystemName VARCHAR(100) NOT NULL,
    ->     Description VARCHAR(255),
    ->     CriticalityLevel VARCHAR(20),
    ->     AssetID INT NOT NULL,
    ->     FOREIGN KEY (AssetID) REFERENCES ASSET(AssetID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DESCRIBE SUBSYSTEM;
+------------------+--------------+------+-----+---------+-------+
| Field            | Type         | Null | Key | Default | Extra |
+------------------+--------------+------+-----+---------+-------+
| SubsystemID      | int          | NO   | PRI | NULL    |       |
| SubsystemName    | varchar(100) | NO   |     | NULL    |       |
| Description      | varchar(255) | YES  |     | NULL    |       |
| CriticalityLevel | varchar(20)  | YES  |     | NULL    |       |
| AssetID          | int          | NO   | MUL | NULL    |       |
+------------------+--------------+------+-----+---------+-------+
5 rows in set (0.01 sec)
```

5.13 Table COMPONENT:

Code:

```sql
CREATE TABLE COMPONENT (
    ComponentID INT PRIMARY KEY,
    ComponentName VARCHAR(100) NOT NULL,
    Description VARCHAR(255),
    CriticalityLevel VARCHAR(20),
    IsRepairable VARCHAR(5),
    SubsystemID INT NOT NULL,

    FOREIGN KEY (SubsystemID)
        REFERENCES SUBSYSTEM(SubsystemID)
);
```

Screenshot:

```
mysql> CREATE TABLE COMPONENT (
    ->     ComponentID INT PRIMARY KEY,
    ->     ComponentName VARCHAR(100),
    ->     Description VARCHAR(255),
    ->     CriticalityLevel VARCHAR(20),
    ->     IsRepairable VARCHAR(5),
    ->     SubsystemID INT NOT NULL,
    ->     FOREIGN KEY (SubsystemID) REFERENCES SUBSYSTEM(SubsystemID
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DESCRIBE COMPONENT;
+------------------+--------------+------+-----+---------+-------+
| Field            | Type         | Null | Key | Default | Extra |
+------------------+--------------+------+-----+---------+-------+
| ComponentID      | int          | NO   | PRI | NULL    |       |
| ComponentName    | varchar(100) | YES  |     | NULL    |       |
| Description      | varchar(255) | YES  |     | NULL    |       |
| CriticalityLevel | varchar(20)  | YES  |     | NULL    |       |
| IsRepairable     | varchar(5)   | YES  |     | NULL    |       |
| SubsystemID      | int          | NO   | MUL | NULL    |       |
+------------------+--------------+------+-----+---------+-------+
6 rows in set (0.01 sec)
```

5.14 Table COMPONENT_DEPENDENCY:

Code:

```sql
CREATE TABLE COMPONENT_DEPENDENCY (
    ComponentID INT NOT NULL,
    DependsOnComponentID INT NOT NULL,

    PRIMARY KEY (ComponentID, DependsOnComponentID),

    FOREIGN KEY (ComponentID)
        REFERENCES COMPONENT(ComponentID),

    FOREIGN KEY (DependsOnComponentID)
        REFERENCES COMPONENT(ComponentID),

    CHECK (ComponentID <> DependsOnComponentID)
);
```

Screenshot:

```
mysql> CREATE TABLE COMPONENT_DEPENDENCY (
    ->     ComponentID INT NOT NULL,
    ->     DependsOnComponentID INT NOT NULL,
    ->     PRIMARY KEY (ComponentID, DependsOnComponentID),
    ->     FOREIGN KEY (ComponentID) REFERENCES COMPONENT(ComponentID),
    ->     FOREIGN KEY (DependsOnComponentID) REFERENCES COMPONENT(Compo
nentID),
    ->     CHECK (ComponentID <> DependsOnComponentID)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> DESCRIBE COMPONENT_DEPENDENCY;
+---------------------+------+------+-----+---------+-------+
| Field               | Type | Null | Key | Default | Extra |
+---------------------+------+------+-----+---------+-------+
| ComponentID         | int  | NO   | PRI | NULL    |       |
| DependsOnComponentID | int  | NO   | PRI | NULL    |       |
+---------------------+------+------+-----+---------+-------+
2 rows in set (0.01 sec)
```

5.15 Table SPARE_PART:

Code:

```sql
CREATE TABLE SPARE_PART (
    SparePartID INT PRIMARY KEY,
    PartName VARCHAR(100) NOT NULL,
    PartNumber VARCHAR(100),
    UOM VARCHAR(30),
    StockQuantity INT DEFAULT 0,
    ReorderLevel INT DEFAULT 0,
    UnitCost DECIMAL(12,2),
    CriticalityLevel VARCHAR(20)
);
```

Screenshot:

```
mysql> CREATE TABLE SPARE_PART (
    ->     SparePartID INT PRIMARY KEY,
    ->     PartName VARCHAR(100) NOT NULL,
    ->     PartNumber VARCHAR(100),
    ->     UOM VARCHAR(30),
    ->     StockQuantity INT DEFAULT 0,
    ->     ReorderLevel INT DEFAULT 0,
    ->     UnitCost DECIMAL(12,2),
    ->     CriticalityLevel VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DESCRIBE SPARE_PART;
+------------------+---------------+------+-----+---------+-------+
| Field            | Type          | Null | Key | Default | Extra |
+------------------+---------------+------+-----+---------+-------+
| SparePartID      | int           | NO   | PRI | NULL    |       |
| PartName         | varchar(100)  | NO   |     | NULL    |       |
| PartNumber       | varchar(100)  | YES  |     | NULL    |       |
| UOM              | varchar(30)   | YES  |     | NULL    |       |
| StockQuantity    | int           | YES  |     | 0       |       |
| ReorderLevel     | int           | YES  |     | 0       |       |
| UnitCost         | decimal(12,2) | YES  |     | NULL    |       |
| CriticalityLevel | varchar(20)   | YES  |     | NULL    |       |
+------------------+---------------+------+-----+---------+-------+
8 rows in set (0.00 sec)
```

5.16 Table COMPONENT_SPARE:

Code:

```sql
CREATE TABLE COMPONENT_SPARE (
    ComponentID INT NOT NULL,
    SparePartID INT NOT NULL,
    QuantityRequired INT,
    IsCritical VARCHAR(5),

    PRIMARY KEY (ComponentID, SparePartID),

    FOREIGN KEY (ComponentID)
        REFERENCES COMPONENT(ComponentID),

    FOREIGN KEY (SparePartID)
        REFERENCES SPARE_PART(SparePartID)
);
```

Screenshot:

```
mysql> CREATE TABLE COMPONENT_SPARE (
    ->     ComponentID INT NOT NULL,
    ->     SparePartID INT NOT NULL,
    ->     QuantityRequired INT,
    ->     IsCritical VARCHAR(5),
    ->     PRIMARY KEY (ComponentID, SparePartID),
    ->     FOREIGN KEY (ComponentID) REFERENCES COMPONENT(ComponentID),
    ->     FOREIGN KEY (SparePartID) REFERENCES SPARE_PART(SparePartID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DESCRIBE COMPONENT_SPARE;
+------------------+------------+------+-----+---------+-------+
| Field            | Type       | Null | Key | Default | Extra |
+------------------+------------+------+-----+---------+-------+
| ComponentID      | int        | NO   | PRI | NULL    |       |
| SparePartID      | int        | NO   | PRI | NULL    |       |
| QuantityRequired | int        | YES  |     | NULL    |       |
| IsCritical       | varchar(5) | YES  |     | NULL    |       |
+------------------+------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

5.17 Table SUPPLIER:

Code:

```sql
CREATE TABLE SUPPLIER (
    SupplierID INT PRIMARY KEY,
    SupplierName VARCHAR(100) NOT NULL,
    ContactPerson VARCHAR(100),
    Phone VARCHAR(20),
    Email VARCHAR(100),
    Address VARCHAR(255)
);
```

Screenshot:

```
mysql> CREATE TABLE SUPPLIER (
    ->     SupplierID INT PRIMARY KEY,
    ->     SupplierName VARCHAR(100) NOT NULL,
    ->     ContactPerson VARCHAR(100),
    ->     Phone VARCHAR(20),
    ->     Email VARCHAR(100),
    ->     Address VARCHAR(255)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> DESCRIBE SUPPLIER;
+---------------+--------------+------+-----+---------+-------+
| Field         | Type         | Null | Key | Default | Extra |
+---------------+--------------+------+-----+---------+-------+
| SupplierID    | int          | NO   | PRI | NULL    |       |
| SupplierName  | varchar(100) | NO   |     | NULL    |       |
| ContactPerson | varchar(100) | YES  |     | NULL    |       |
| Phone         | varchar(20)  | YES  |     | NULL    |       |
| Email         | varchar(100) | YES  |     | NULL    |       |
| Address       | varchar(255) | YES  |     | NULL    |       |
+---------------+--------------+------+-----+---------+-------+
6 rows in set (0.00 sec)
```

5.18 Table SUPPLIER_SPARE:

Code:

```
CREATE TABLE SUPPLIER_SPARE (
    SupplierID INT NOT NULL,
    SparePartID INT NOT NULL,
    LeadTimeDay INT,
    UnitPrice DECIMAL(12,2),
    PreferredSupplier VARCHAR(5),

    PRIMARY KEY (SupplierID, SparePartID),

    FOREIGN KEY (SupplierID)
        REFERENCES SUPPLIER(SupplierID),

    FOREIGN KEY (SparePartID)
        REFERENCES SPARE_PART(SparePartID)
);
```

Screenshot:

```
mysql> CREATE TABLE SUPPLIER_SPARE (
    ->     SupplierID INT NOT NULL,
    ->     SparePartID INT NOT NULL,
    ->     LeadTimeDay INT,
    ->     UnitPrice DECIMAL(12,2),
    ->     PreferredSupplier VARCHAR(5),
    ->     PRIMARY KEY (SupplierID, SparePartID),
    ->     FOREIGN KEY (SupplierID) REFERENCES SUPPLIER(SupplierID),
    ->     FOREIGN KEY (SparePartID) REFERENCES SPARE_PART(SparePartID
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DESCRIBE SUPPLIER_SPARE;
+------------------+---------------+------+-----+---------+-------+
| Field            | Type          | Null | Key | Default | Extra |
+------------------+---------------+------+-----+---------+-------+
| SupplierID       | int           | NO   | PRI | NULL    |       |
| SparePartID      | int           | NO   | PRI | NULL    |       |
| LeadTimeDay      | int           | YES  |     | NULL    |       |
| UnitPrice        | decimal(12,2) | YES  |     | NULL    |       |
| PreferredSupplier | varchar(5)    | YES  |     | NULL    |       |
+------------------+---------------+------+-----+---------+-------+
5 rows in set (0.01 sec)
```

6. DML STATEMENTS

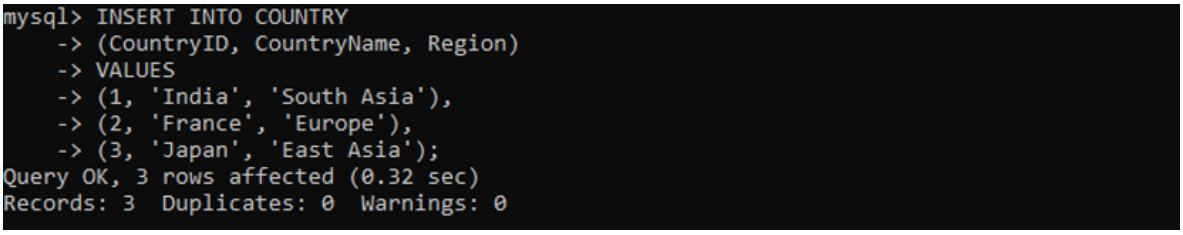
Data Manipulation Language statements are used to insert, retrieve and modify records. The project SQL scripts and database export contain the schema and data preparation materials. The corrected seed script is separate from the schema export. Use the actual successful MySQL Workbench/CLI execution results as evidence; do not use a planned or expected output as proof.

6.1 Table COUNTRY:

Code:

```
INSERT INTO COUNTRY
(CountryID, CountryName, Region)
VALUES
(1, 'India', 'South Asia'),
(2, 'France', 'Europe'),
(3, 'Japan', 'East Asia');
```

Screenshot:



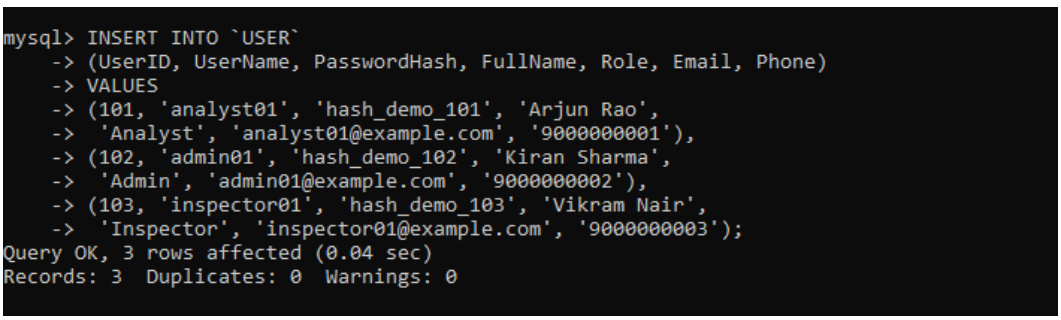
```
mysql> INSERT INTO COUNTRY
    -> (CountryID, CountryName, Region)
    -> VALUES
    -> (1, 'India', 'South Asia'),
    -> (2, 'France', 'Europe'),
    -> (3, 'Japan', 'East Asia');
Query OK, 3 rows affected (0.32 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

6.2 Table USER:

Code:

```
INSERT INTO `USER`
(UserID, UserName, PasswordHash, FullName, Role, Email, Phone)
VALUES
(101, 'analyst01', 'hash_demo_101', 'Arjun Rao',
 'Analyst', 'analyst01@example.com', '9000000001'),
(102, 'admin01', 'hash_demo_102', 'Kiran Sharma',
 'Admin', 'admin01@example.com', '9000000002'),
(103, 'inspector01', 'hash_demo_103', 'Vikram Nair',
 'Inspector', 'inspector01@example.com', '9000000003')
```

Screenshot:



```
mysql> INSERT INTO `USER`
    -> (UserID, UserName, PasswordHash, FullName, Role, Email, Phone)
    -> VALUES
    -> (101, 'analyst01', 'hash_demo_101', 'Arjun Rao',
    ->  'Analyst', 'analyst01@example.com', '9000000001'),
    -> (102, 'admin01', 'hash_demo_102', 'Kiran Sharma',
    ->  'Admin', 'admin01@example.com', '9000000002'),
    -> (103, 'inspector01', 'hash_demo_103', 'Vikram Nair',
    ->  'Inspector', 'inspector01@example.com', '9000000003');
Query OK, 3 rows affected (0.04 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

6.3 Table SERVICE:

Code:

```
INSERT INTO SERVICE
(ServiceID, ServiceName, Description, CountryID)
VALUES
(1, 'Indian Air Force', 'Air service of India', 1),
(2, 'Indian Navy', 'Naval service of India', 1),
(3, 'Indian Army', 'Land service of India', 1);
```

Screenshot:

```
mysql> INSERT INTO SERVICE
    -> (ServiceID, ServiceName, Description, CountryID)
    -> VALUES
    -> (1, 'Indian Air Force', 'Air service of India', 1),
    -> (2, 'Indian Navy', 'Naval service of India', 1),
    -> (3, 'Indian Army', 'Land service of India', 1);
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

6.4 Table DEFENCE_INDICATOR:

Code:

```
INSERT INTO DEFENCE_INDICATOR
(IndicatorID, CountryID, Year, DefenceBudget,
 RevenueExpenditure, CapitalExp, MaintenanceExp,
 IndigenisationIndex, Notes)
VALUES
(1001, 1, 2024, 620000.00, 360000.00, 260000.00,
 85000.00, 68.50, 'Synthetic demonstration data'),
(1002, 1, 2025, 680000.00, 380000.00, 300000.00,
 92000.00, 72.00, 'Synthetic demonstration data'),
(1003, 2, 2025, 610000.00, 350000.00, 260000.00,
 88000.00, 70.00, 'Synthetic demonstration data'),
(1004, 3, 2025, 540000.00, 330000.00, 210000.00,
 76000.00, 65.00, 'Synthetic demonstration data');
```

Screenshot:

```
mysql> INSERT INTO DEFENCE_INDICATOR
    -> (IndicatorID, CountryID, Year, DefenceBudget,
    ->  RevenueExpenditure, CapitalExp, MaintenanceExp,
    ->  IndigenisationIndex, Notes)
    -> VALUES
    -> (1001, 1, 2024, 620000.00, 360000.00, 260000.00,
    ->  85000.00, 68.50, 'Synthetic demonstration data'),
    -> (1002, 1, 2025, 680000.00, 380000.00, 300000.00,
    ->  92000.00, 72.00, 'Synthetic demonstration data'),
    -> (1003, 2, 2025, 610000.00, 350000.00, 260000.00,
    ->  88000.00, 70.00, 'Synthetic demonstration data'),
    -> (1004, 3, 2025, 540000.00, 330000.00, 210000.00,
    ->  76000.00, 65.00, 'Synthetic demonstration data');
Query OK, 4 rows affected (0.03 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

6.5 Table UNIT:

Code:

```
INSERT INTO UNIT
(UnitID, UnitName, Type, Location, ServiceID)
VALUES
(201, 'Fighter Squadron Alpha', 'Formation', 'Bengaluru', 1),
(202, 'Transport Squadron Bravo', 'Formation', 'Chandigarh', 1),
(203, 'Naval Support Unit', 'Base', 'Kochi', 2),
(204, 'Armoured Unit Charlie', 'Formation', 'Jaisalmer', 3);
```

Screenshot:

```
mysql> INSERT INTO UNIT
    -> (UnitID, UnitName, Type, Location, ServiceID)
    -> VALUES
    -> (201, 'Fighter Squadron Alpha', 'Formation', 'Bengaluru', 1),
    -> (202, 'Transport Squadron Bravo', 'Formation', 'Chandigarh', 1),
    -> (203, 'Naval Support Unit', 'Base', 'Kochi', 2),
    -> (204, 'Armoured Unit Charlie', 'Formation', 'Jaisalmer', 3);
Query OK, 4 rows affected (0.04 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

6.6 Table ASSET_TYPE:

Code:

```
INSERT INTO ASSET_TYPE
(AssetTypeID, AssetTypeName, Category, Description)
VALUES
(301, 'Fighter Aircraft', 'Aircraft',
 'Synthetic fighter aircraft asset category'),
(302, 'Transport Aircraft', 'Aircraft',
 'Synthetic transport aircraft category'),
(303, 'Naval Support Platform', 'Naval',
 'Synthetic naval support asset category'),
(304, 'Armoured Vehicle', 'Land',
 'Synthetic armoured vehicle category');
```

Screenshot:

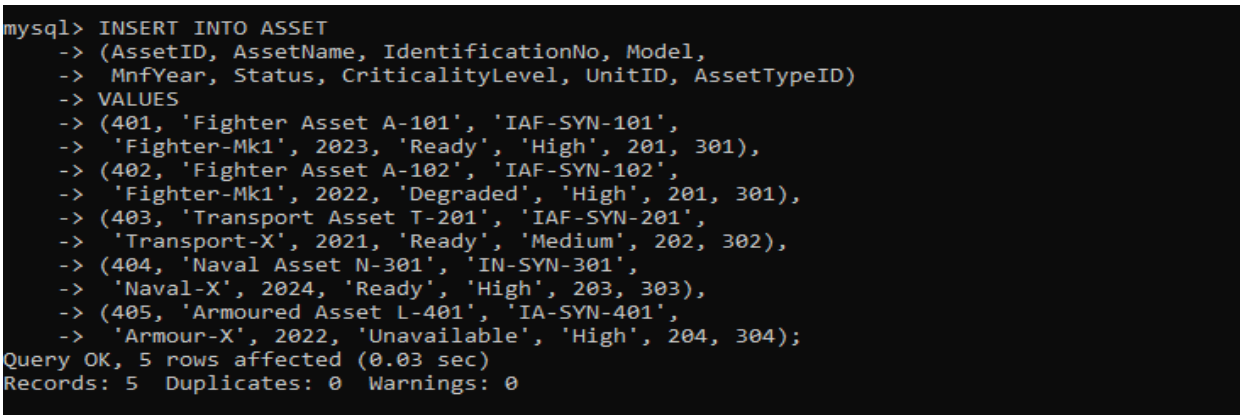
```
mysql> INSERT INTO ASSET_TYPE
    -> (AssetTypeID, AssetTypeName, Category, Description)
    -> VALUES
    -> (301, 'Fighter Aircraft', 'Aircraft',
    ->  'Synthetic fighter aircraft asset category'),
    -> (302, 'Transport Aircraft', 'Aircraft',
    ->  'Synthetic transport aircraft category'),
    -> (303, 'Naval Support Platform', 'Naval',
    ->  'Synthetic naval support asset category'),
    -> (304, 'Armoured Vehicle', 'Land',
    ->  'Synthetic armoured vehicle category');
Query OK, 4 rows affected (0.04 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

6.7 Table ASSET:

Code:

```
INSERT INTO ASSET
(AssetID, AssetName, IdentificationNo, Model,
 MnfYear, Status, CriticalityLevel, UnitID, AssetTypeID)
VALUES
(401, 'Fighter Asset A-101', 'IAF-SYN-101',
 'Fighter-Mk1', 2023, 'Ready', 'High', 201, 301),
(402, 'Fighter Asset A-102', 'IAF-SYN-102',
 'Fighter-Mk1', 2022, 'Degraded', 'High', 201, 301),
(403, 'Transport Asset T-201', 'IAF-SYN-201',
 'Transport-X', 2021, 'Ready', 'Medium', 202, 302),
(404, 'Naval Asset N-301', 'IN-SYN-301',
 'Naval-X', 2024, 'Ready', 'High', 203, 303),
(405, 'Armoured Asset L-401', 'IA-SYN-401',
 'Armour-X', 2022, 'Unavailable', 'High', 204, 304);
```

Screenshot:



6.8 Table FAILURE_EVENT:

Code:

```
INSERT INTO FAILURE_EVENT
(FailureID, FailureType, Severity, OccurredAt,
 Description, AssetID)
VALUES
(501, 'Cooling System Failure', 'High',
 '2026-09-20 10:30:00',
 'Abnormal cooling performance detected', 402),
(502, 'Hydraulic Failure', 'Medium',
 '2026-09-21 14:15:00',
 'Hydraulic pressure below expected range', 405),
(503, 'Power System Failure', 'High',
 '2026-09-23 09:45:00',
 'Power subsystem malfunction detected', 401);
```

Screenshot:

```
mysql> INSERT INTO FAILURE_EVENT
    -> (FailureID, FailureType, Severity, OccurredAt,
    ->  Description, AssetID)
    -> VALUES
    -> (501, 'Cooling System Failure', 'High',
    ->  '2026-09-20 10:30:00',
    ->  'Abnormal cooling performance detected', 402),
    -> (502, 'Hydraulic Failure', 'Medium',
    ->  '2026-09-21 14:15:00',
    ->  'Hydraulic pressure below expected range', 405),
    -> (503, 'Power System Failure', 'High',
    ->  '2026-09-23 09:45:00',
    ->  'Power subsystem malfunction detected', 401);
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

6.9 Table INSPECTION:

Code:

```
INSERT INTO INSPECTION
(InspectionID, FailureID, InspectorID,
 InspectionType, InspectionDate, Result, Remarks)
VALUES
(601, 501, 103,
 'Technical Inspection', '2026-09-20',
 'Fail', 'Cooling component requires replacement'),
(602, 502, 103,
 'Technical Inspection', '2026-09-21',
 'Pass', 'Temporary issue identified'),
(603, 503, 103,
 'Technical Inspection', '2026-09-23',
 'Fail', 'Power component requires maintenance');
        REFERENCES FAILURE_EVENT(FailureID),

    FOREIGN KEY (InspectorID)
        REFERENCES `USER`(UserID)
);
```

Screenshot:

```
mysql> INSERT INTO INSPECTION
    -> (InspectionID, FailureID, InspectorID,
    ->  InspectionType, InspectionDate, Result, Remarks)
    -> VALUES
    -> (601, 501, 103,
    ->  'Technical Inspection', '2026-09-20',
    ->  'Fail', 'Cooling component requires replacement'),
    -> (602, 502, 103,
    ->  'Technical Inspection', '2026-09-21',
    ->  'Pass', 'Temporary issue identified'),
    -> (603, 503, 103,
    ->  'Technical Inspection', '2026-09-23',
    ->  'Fail', 'Power component requires maintenance');
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

6.10 Table MAINTENANCE_WORK_ORDER:

Code:

```
INSERT INTO MAINTENANCE_WORK_ORDER
(WorkOrderID, AssetID, Description,
 OrderDate, Priority, Status)
VALUES
(701, 402,
 'Repair cooling system',
 '2026-09-20', 'High', 'InProgress'),
(702, 405,
 'Inspect hydraulic system',
 '2026-09-21', 'Medium', 'Closed'),
(703, 401,
 'Repair power subsystem',
 '2026-09-23', 'High', 'Open');
```

Screenshot:

```
mysql> INSERT INTO MAINTENANCE_WORK_ORDER
    -> (WorkOrderID, AssetID, Description,
    ->  OrderDate, Priority, Status)
    -> VALUES
    -> (701, 402,
    ->  'Repair cooling system',
    ->  '2026-09-20', 'High', 'InProgress'),
    -> (702, 405,
    ->  'Inspect hydraulic system',
    ->  '2026-09-21', 'Medium', 'Closed'),
    -> (703, 401,
    ->  'Repair power subsystem',
    ->  '2026-09-23', 'High', 'Open');
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

6.11 Table MAINTENANCE_EVENT:

Code:

```
INSERT INTO MAINTENANCE_EVENT
(MaintenanceID, WorkOrderID, Remarks,
 PerformedBy, StartDateTime, EndDateTime,
 MaintenanceType, ActionTaken,
 DowntimeHours, CostIncurred)
VALUES
(801, 702,'Hydraulic inspection completed', 103, '2026-09-21 16:00:00', '2026-09-21
18:00:00',
 'Inspection','Hydraulic system inspected',2.00,25000.00),
(802, 701,'Cooling system repair initiated',103,'2026-09-22 09:00:00','2026-09-22 15:30:00',
 'Repair','Cooling component replaced',6.50,85000.00);
        REFERENCES `USER`(UserID)
);
```

Screenshot:

```
mysql> INSERT INTO MAINTENANCE_EVENT
    -> (MaintenanceID, WorkOrderID, Remarks,
    ->  PerformedBy, StartDateTime, EndDateTime,
    ->  MaintenanceType, ActionTaken,
    ->  DowntimeHours, CostIncurred)
    -> VALUES
    -> (801, 702,'Hydraulic inspection completed', 103, '2026-09-21 16:00:00', '2026-09-21 18:00:00',
    ->  'Inspection','Hydraulic system inspected',2.00,25000.00),
    -> (802, 701,'Cooling system repair initiated',103,'2026-09-22 09:00:00','2026-09-22 15:30:00',
    ->  'Repair','Cooling component replaced',6.50,85000.00);
Query OK, 2 rows affected (0.03 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

6.12 Table SUBSYSTEM:

Code:

```
INSERT INTO SUBSYSTEM
(SubsystemID, SubsystemName, Description,
 CriticalityLevel, AssetID)
VALUES
(901, 'Engine System',
 'Primary propulsion subsystem',
 'High', 401),
(902, 'Cooling System',
 'Thermal management subsystem',
 'High', 402),
(903, 'Hydraulic System',
 'Hydraulic control subsystem',
 'High', 405),
(904, 'Power System',
 'Electrical power subsystem',
 'High', 401);
```

Screenshot:

```
mysql> INSERT INTO SUBSYSTEM
    -> (SubsystemID, SubsystemName, Description,
    ->  CriticalityLevel, AssetID)
    -> VALUES
    -> (901, 'Engine System',
    ->  'Primary propulsion subsystem',
    ->  'High', 401),
    -> (902, 'Cooling System',
    ->  'Thermal management subsystem',
    ->  'High', 402),
    -> (903, 'Hydraulic System',
    ->  'Hydraulic control subsystem',
    ->  'High', 405),
    -> (904, 'Power System',
    ->  'Electrical power subsystem',
    ->  'High', 401);
Query OK, 4 rows affected (0.03 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

6.13 Table COMPONENT:

Code:

```
INSERT INTO COMPONENT
(ComponentID, ComponentName, Description,
 CriticalityLevel, IsRepairable, SubsystemID)
VALUES
(1001, 'Fuel Pump',
 'Synthetic fuel delivery component',
 'High', 'Yes', 901),
(1002, 'Engine Control Unit',
 'Synthetic engine control component',
 'High', 'Yes', 901),
(1003, 'Cooling Pump',
 'Synthetic cooling component',
 'High', 'Yes', 902),
(1004, 'Hydraulic Valve',
 'Synthetic hydraulic control component',
 'High', 'Yes', 903),
(1005, 'Power Controller',
 'Synthetic electrical control component',
 'High', 'Yes', 904);
```

Screenshot:

```
mysql> INSERT INTO COMPONENT
    -> (ComponentID, ComponentName, Description,
    ->  CriticalityLevel, IsRepairable, SubsystemID)
    -> VALUES
    -> (1001, 'Fuel Pump',
    ->  'Synthetic fuel delivery component',
    ->  'High', 'Yes', 901),
    -> (1002, 'Engine Control Unit',
    ->  'Synthetic engine control component',
    ->  'High', 'Yes', 901),
    -> (1003, 'Cooling Pump',
    ->  'Synthetic cooling component',
    ->  'High', 'Yes', 902),
    -> (1004, 'Hydraulic Valve',
    ->  'Synthetic hydraulic control component',
    ->  'High', 'Yes', 903),
    -> (1005, 'Power Controller',
    ->  'Synthetic electrical control component',
    ->  'High', 'Yes', 904);
Query OK, 5 rows affected (0.03 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

6.14 Table COMPONENT_DEPENDENCY:

Code:

```
INSERT INTO COMPONENT_DEPENDENCY
(ComponentID, DependsOnComponentID)
VALUES
(1002, 1001),
(1003, 1001),
(1005, 1002);
```

Screenshot:

```
mysql> INSERT INTO COMPONENT_DEPENDENCY
    -> (ComponentID, DependsOnComponentID)
    -> VALUES
    -> (1002, 1001),
    -> (1003, 1001),
    -> (1005, 1002);
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

6.15 Table SPARE_PART:

Code:

```
INSERT INTO SPARE_PART
(SparePartID, PartName, PartNumber,
 UOM, StockQuantity, ReorderLevel,
 UnitCost, CriticalityLevel)
VALUES
(1101, 'Fuel Pump Seal', 'FPS-001',
 'Piece', 50, 10, 5000.00, 'High'),
(1102, 'Cooling Pump Assembly', 'CPA-002',
 'Piece', 8, 3, 85000.00, 'High'),
(1103, 'Hydraulic Valve Kit', 'HVK-003',
 'Kit', 15, 5, 25000.00, 'Medium'),
(1104, 'Power Controller Module', 'PCM-004',
 'Piece', 6, 2, 120000.00, 'High');
```

Screenshot:

```
mysql> INSERT INTO SPARE_PART
    -> (SparePartID, PartName, PartNumber,
    ->  UOM, StockQuantity, ReorderLevel,
    ->  UnitCost, CriticalityLevel)
    -> VALUES
    -> (1101, 'Fuel Pump Seal', 'FPS-001',
    ->  'Piece', 50, 10, 5000.00, 'High'),
    -> (1102, 'Cooling Pump Assembly', 'CPA-002',
    ->  'Piece', 8, 3, 85000.00, 'High'),
    -> (1103, 'Hydraulic Valve Kit', 'HVK-003',
    ->  'Kit', 15, 5, 25000.00, 'Medium'),
    -> (1104, 'Power Controller Module', 'PCM-004',
    ->  'Piece', 6, 2, 120000.00, 'High');
Query OK, 4 rows affected (0.03 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

6.16 Table COMPONENT_SPARE:

Code:

```
INSERT INTO COMPONENT_SPARE
(ComponentID, SparePartID, QuantityRequired, IsCritical)
VALUES
(1001, 1101, 2, 'Yes'),
(1003, 1102, 1, 'Yes'),
(1004, 1103, 1, 'Yes'),
(1005, 1104, 1, 'Yes');
```

Screenshot:

```
mysql> INSERT INTO COMPONENT_SPARE
    -> (ComponentID, SparePartID, QuantityRequired, IsCritical)
    -> VALUES
    -> (1001, 1101, 2, 'Yes'),
    -> (1003, 1102, 1, 'Yes'),
    -> (1004, 1103, 1, 'Yes'),
    -> (1005, 1104, 1, 'Yes');
Query OK, 4 rows affected (0.03 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

6.17 Table SUPPLIER:

Code:

```
INSERT INTO SUPPLIER
(SupplierID, SupplierName, ContactPerson,
 Phone, Email, Address)
VALUES
(1201, 'Bharat Defence Components',
 'Rohan Mehta',
 '9000012001',
 'supplier1@example.com',
 'Bengaluru'),

(1202, 'Aero Systems Manufacturing',
 'Neha Rao',
 '9000012002',
 'supplier2@example.com',
 'Hyderabad'),
(1203, 'Precision Engineering Works',
 'Amit Kumar',
 '9000012003',
 'supplier3@example.com',
 'Pune');
```

Screenshot:

```
mysql> INSERT INTO SUPPLIER
    -> (SupplierID, SupplierName, ContactPerson,
    ->  Phone, Email, Address)
    -> VALUES
    -> (1201, 'Bharat Defence Components',
    ->  'Rohan Mehta',
    ->  '9000012001',
    ->  'supplier1@example.com',
    ->  'Bengaluru'),
    -> (1202, 'Aero Systems Manufacturing',
    ->  'Neha Rao',
    ->  '9000012002',
    ->  'supplier2@example.com',
    ->  'Hyderabad'),
    -> (1203, 'Precision Engineering Works',
    ->  'Amit Kumar',
    ->  '9000012003',
    ->  'supplier3@example.com',
    ->  'Pune');
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

6.18 Table SUPPLIER_SPARE:

Code:

```
INSERT INTO SUPPLIER_SPARE
(SupplierID, SparePartID, LeadTimeDay,
 UnitPrice, PreferredSupplier)
VALUES
(1201, 1101, 7, 5000.00, 'Yes'),
(1202, 1102, 14, 85000.00, 'Yes'),
(1203, 1103, 10, 25000.00, 'No'),
(1202, 1104, 21, 120000.00, 'Yes');
```

Screenshot:

```
mysql> INSERT INTO SUPPLIER_SPARE
    -> (SupplierID, SparePartID, LeadTimeDay,
    ->  UnitPrice, PreferredSupplier)
    -> VALUES
    -> (1201, 1101, 7, 5000.00, 'Yes'),
    -> (1202, 1102, 14, 85000.00, 'Yes'),
    -> (1203, 1103, 10, 25000.00, 'No'),
    -> (1202, 1104, 21, 120000.00, 'Yes');
Query OK, 4 rows affected (0.03 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

7. QUERIES AND RELATIONAL OPERATIONS

7.1 Code:

```
/*==============
   PART 1: UPDATE
===============*/
-- 1. Update asset status

UPDATE ASSET
SET Status = 'Degraded'
WHERE AssetID = 401;


-- 2. Update maintenance work order

UPDATE MAINTENANCE_WORK_ORDER
SET Status = 'Completed'
WHERE WorkOrderID = 701;


-- 3. Update spare stock

UPDATE SPARE_PART
SET StockQuantity = StockQuantity - 2
WHERE SparePartID = 1101;


-- 4. Update supplier preference

UPDATE SUPPLIER_SPARE
SET PreferredSupplier = 'Yes'
WHERE SupplierID = 1203
AND SparePartID = 1103;


/* ===============
   PART 2: DELETE
   ================*/

Delete a dependency relationship.
DELETE FROM COMPONENT_DEPENDENCY
WHERE ComponentID = 1005
AND DependsOnComponentID = 1002;


   Delete a supplier-spare relationship.
```

```sql
DELETE FROM SUPPLIER_SPARE
WHERE SupplierID = 1203
AND SparePartID = 1103;


/* ============================
   PART 3: BASIC SELECT QUERIES
   ============================*/

-- 1. Display all assets
SELECT *
FROM ASSET;


-- 2. Display only unavailable assets
SELECT AssetID, AssetName, Status
FROM ASSET
WHERE Status = 'Unavailable';


-- 3. Display high-criticality assets
SELECT AssetID, AssetName, CriticalityLevel
FROM ASSET
WHERE CriticalityLevel = 'High';


-- 4. Display assets ordered by manufacturing year
SELECT AssetName, MnfYear
FROM ASSET
ORDER BY MnfYear DESC;


-- 5. Display different asset statuses
SELECT DISTINCT Status
FROM ASSET;


/* ============================
   PART 4: AGGREGATE FUNCTIONS
   ============================*/


-- 1. Count total assets
SELECT COUNT(*) AS TotalAssets
FROM ASSET;


-- 2. Average maintenance cost
SELECT AVG(CostIncurred) AS AverageMaintenanceCost
FROM MAINTENANCE_EVENT;
```

```sql
-- 3. Total maintenance cost
SELECT SUM(CostIncurred) AS TotalMaintenanceCost
FROM MAINTENANCE_EVENT;


-- 4. Maximum downtime
SELECT MAX(DowntimeHours) AS MaximumDowntime
FROM MAINTENANCE_EVENT;


-- 5. Minimum maintenance cost
SELECT MIN(CostIncurred) AS MinimumMaintenanceCost
FROM MAINTENANCE_EVENT;


/* ====================
   PART 5: GROUP BY
   ====================*/

-- Number of assets by status
SELECT Status, COUNT(*) AS AssetCount
FROM ASSET
GROUP BY Status;


-- Number of assets by criticality
SELECT CriticalityLevel, COUNT(*) AS AssetCount
FROM ASSET
GROUP BY CriticalityLevel;


-- Total maintenance cost by maintenance type
SELECT MaintenanceType,
       SUM(CostIncurred) AS TotalCost
FROM MAINTENANCE_EVENT
GROUP BY MaintenanceType;


/* ========================
   PART 6: HAVING
   ========================*/


-- Asset statuses having more than one asset
SELECT Status, COUNT(*) AS AssetCount
FROM ASSET
GROUP BY Status
HAVING COUNT(*) > 1;


-- Maintenance types with total cost greater than 50000
```

```sql
SELECT MaintenanceType,
       SUM(CostIncurred) AS TotalCost
FROM MAINTENANCE_EVENT
GROUP BY MaintenanceType
HAVING SUM(CostIncurred) > 50000;


/* ======================
   PART 7: JOINS
   ======================*/


-- 1. Asset + Unit
SELECT
    a.AssetName,
    u.UnitName,
    u.Location
FROM ASSET a
JOIN UNIT u
    ON a.UnitID = u.UnitID;


-- 2. Asset + Unit + Service
SELECT
    a.AssetName,
    u.UnitName,
    s.ServiceName
FROM ASSET a
JOIN UNIT u
    ON a.UnitID = u.UnitID
JOIN SERVICE s
    ON u.ServiceID = s.ServiceID;


-- 3. Asset + Subsystem
SELECT
    a.AssetName,
    ss.SubsystemName,
    ss.CriticalityLevel
FROM ASSET a
JOIN SUBSYSTEM ss
    ON a.AssetID = ss.AssetID;


-- 4. Subsystem + Component
SELECT
    ss.SubsystemName,
    c.ComponentName,
    c.CriticalityLevel
FROM SUBSYSTEM ss
```

```sql
JOIN COMPONENT c
    ON ss.SubsystemID = c.SubsystemID;


-- 5. Component + Spare Part
SELECT
    c.ComponentName,
    sp.PartName,
    cs.QuantityRequired,
    cs.IsCritical
FROM COMPONENT c
JOIN COMPONENT_SPARE cs
    ON c.ComponentID = cs.ComponentID
JOIN SPARE_PART sp
    ON cs.SparePartID = sp.SparePartID;


-- 6. Spare Part + Supplier
SELECT
    sp.PartName,
    su.SupplierName,
    ss.UnitPrice,
    ss.LeadTimeDay,
    ss.PreferredSupplier
FROM SPARE_PART sp
JOIN SUPPLIER_SPARE ss
    ON sp.SparePartID = ss.SparePartID
JOIN SUPPLIER su
    ON ss.SupplierID = su.SupplierID;


-- 7. Maintenance + User
SELECT
    me.MaintenanceID,
    me.MaintenanceType,
    u.FullName AS PerformedBy,
    me.CostIncurred
FROM MAINTENANCE_EVENT me
JOIN `USER` u
    ON me.PerformedBy = u.UserID;


/* ============================
   PART 8: MULTIPLE-TABLE JOIN
   ============================*/


-- Show complete asset hierarchy
SELECT
    s.ServiceName,
```

```sql
    u.UnitName,
    a.AssetName,
    ss.SubsystemName,
    c.ComponentName
FROM SERVICE s
JOIN UNIT u
    ON s.ServiceID = u.ServiceID
JOIN ASSET a
    ON u.UnitID = a.UnitID
JOIN SUBSYSTEM ss
    ON a.AssetID = ss.AssetID
JOIN COMPONENT c
    ON ss.SubsystemID = c.SubsystemID;


/* ======================
   PART 9: SUBQUERIES
   ======================*/


-- 1. Assets having above-average maintenance cost
SELECT DISTINCT a.AssetID, a.AssetName
FROM ASSET a
JOIN MAINTENANCE_WORK_ORDER wo
    ON a.AssetID = wo.AssetID
JOIN MAINTENANCE_EVENT me
    ON wo.WorkOrderID = me.WorkOrderID
WHERE me.CostIncurred >
(
    SELECT AVG(CostIncurred)
    FROM MAINTENANCE_EVENT
);


-- 2. Assets with at least one failure
SELECT AssetID, AssetName
FROM ASSET
WHERE AssetID IN
(
    SELECT AssetID
    FROM FAILURE_EVENT
);


-- 3. Spare parts below reorder level
SELECT SparePartID, PartName,
       StockQuantity, ReorderLevel
FROM SPARE_PART
WHERE StockQuantity <= ReorderLevel;
```

```sql
/* ================================
   PART 10: CORRELATED SUBQUERY
   ================================*/

 --Find spare parts whose stock is below their own reorder level.
SELECT sp1.SparePartID,
       sp1.PartName,
       sp1.StockQuantity,
       sp1.ReorderLevel
FROM SPARE_PART sp1
WHERE sp1.StockQuantity <=
      sp1.ReorderLevel;


/* ============================
   PART 11: SET OPERATIONS
   ============================*/


-- Assets that are Ready or Degraded
SELECT AssetName
FROM ASSET
WHERE Status = 'Ready'
UNION
SELECT AssetName
FROM ASSET
WHERE Status = 'Degraded';
```

Screenshot:

7.2: UPDATE

1. **Update asset status**

```
mysql> select * from asset where AssetID = 401;
+---------+--------------------+------------------+-------------+---------+--------+------------------+--------+-------------+
| AssetID | AssetName          | IdentificationNo | Model       | MnfYear | Status | CriticalityLevel | UnitID | AssetTypeID |
+---------+--------------------+------------------+-------------+---------+--------+------------------+--------+-------------+
|     401 | Fighter Asset A-101 | IAF-SYN-101      | Fighter-Mk1 |    2023 | Ready  | High             |    201 |         301 |
+---------+--------------------+------------------+-------------+---------+--------+------------------+--------+-------------+
1 row in set (0.01 sec)

mysql> UPDATE ASSET SET Status = 'Degraded' WHERE AssetID = 401;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from asset where AssetID = 401;
+---------+--------------------+------------------+-------------+---------+----------+------------------+--------+-------------+
| AssetID | AssetName          | IdentificationNo | Model       | MnfYear | Status   | CriticalityLevel | UnitID | AssetTypeID |
+---------+--------------------+------------------+-------------+---------+----------+------------------+--------+-------------+
|     401 | Fighter Asset A-101 | IAF-SYN-101      | Fighter-Mk1 |    2023 | Degraded | High             |    201 |         301 |
+---------+--------------------+------------------+-------------+---------+----------+------------------+--------+-------------+
1 row in set (0.00 sec)
```

2. Update maintenance work order

```
mysql> select * from MAINTENANCE_WORK_ORDER where WorkOrderID = 701;
+-------------+---------+-----------------------+------------+----------+------------+
| WorkOrderID | AssetID | Description           | OrderDate  | Priority | Status     |
+-------------+---------+-----------------------+------------+----------+------------+
|         701 |     402 | Repair cooling system | 2026-09-20 | High     | InProgress |
+-------------+---------+-----------------------+------------+----------+------------+
1 row in set (0.01 sec)

mysql> UPDATE MAINTENANCE_WORK_ORDER SET Status = 'Completed' WHERE WorkOrderID = 701;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from MAINTENANCE_WORK_ORDER where WorkOrderID = 701;
+-------------+---------+-----------------------+------------+----------+-----------+
| WorkOrderID | AssetID | Description           | OrderDate  | Priority | Status    |
+-------------+---------+-----------------------+------------+----------+-----------+
|         701 |     402 | Repair cooling system | 2026-09-20 | High     | Completed |
+-------------+---------+-----------------------+------------+----------+-----------+
1 row in set (0.00 sec)
```

3. Update Spare part

```
mysql> select * from SUPPLIER_SPARE WHERE SupplierID = 1203 AND SparePartID = 1103;
+------------+-------------+-------------+-----------+--------------------+
| SupplierID | SparePartID | LeadTimeDay | UnitPrice | PreferredSupplier  |
+------------+-------------+-------------+-----------+--------------------+
|       1203 |        1103 |          10 |  25000.00 | No                 |
+------------+-------------+-------------+-----------+--------------------+
1 row in set (0.00 sec)

mysql> UPDATE SUPPLIER_SPARE SET PreferredSupplier = 'Yes' WHERE SupplierID = 1203 AND SparePartID = 1103;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from SUPPLIER_SPARE WHERE SupplierID = 1203 AND SparePartID = 1103;
+------------+-------------+-------------+-----------+--------------------+
| SupplierID | SparePartID | LeadTimeDay | UnitPrice | PreferredSupplier  |
+------------+-------------+-------------+-----------+--------------------+
|       1203 |        1103 |          10 |  25000.00 | Yes                |
+------------+-------------+-------------+-----------+--------------------+
1 row in set (0.01 sec)
```

7.3: DELETE

1. **Delete a dependency relationship.**

```
mysql> select ComponentID from COMPONENT_DEPENDENCY;
+-------------+
| ComponentID |
+-------------+
|        1002 |
|        1003 |
|        1005 |
+-------------+
3 rows in set (0.00 sec)

mysql> DELETE FROM COMPONENT_DEPENDENCY WHERE ComponentID = 1005 AND DependsOnComponentID = 1002;
Query OK, 1 row affected (0.01 sec)

mysql> select ComponentID from COMPONENT_DEPENDENCY;
+-------------+
| ComponentID |
+-------------+
|        1002 |
|        1003 |
+-------------+
2 rows in set (0.00 sec)
```

2. **Delete a supplier-spare relationship.**

```
mysql> select SupplierID from SUPPLIER_SPARE;
+------------+
| SupplierID |
+------------+
|       1201 |
|       1202 |
|       1203 |
|       1202 |
+------------+
4 rows in set (0.00 sec)

mysql> DELETE FROM SUPPLIER_SPARE WHERE SupplierID = 1203 AND SparePartID = 1103;
Query OK, 1 row affected (0.01 sec)

mysql> select SupplierID from SUPPLIER_SPARE;
+------------+
| SupplierID |
+------------+
|       1201 |
|       1202 |
|       1202 |
+------------+
3 rows in set (0.00 sec)
```

7.4: BASIC SELECT QUERIES

1. **Display all assets**

```
mysql> SELECT *FROM ASSET;
+---------+----------------------+-----------------+------------+---------+------------+------------------+---------+-------------+
| AssetID | AssetName            | IdentificationNo | Model      | MnfYear | Status     | CriticalityLevel | UnitID | AssetTypeID |
+---------+----------------------+-----------------+------------+---------+------------+------------------+---------+-------------+
|     401 | Fighter Asset A-101  | IAF-SYN-101     | Fighter-Mk1 |    2023 | Degraded   | High             |    201 |         301 |
|     402 | Fighter Asset A-102  | IAF-SYN-102     | Fighter-Mk1 |    2022 | Degraded   | High             |    201 |         301 |
|     403 | Transport Asset T-201 | IAF-SYN-201     | Transport-X |    2021 | Ready      | Medium           |    202 |         302 |
|     404 | Naval Asset N-301    | IN-SYN-301      | Naval-X    |    2024 | Ready      | High             |    203 |         303 |
|     405 | Armoured Asset L-401 | IA-SYN-401      | Armour-X   |    2022 | Unavailable | High             |    204 |         304 |
+---------+----------------------+-----------------+------------+---------+------------+------------------+---------+-------------+
5 rows in set (0.00 sec)
```

2. **Display only unavailable assets**

```
mysql> SELECT AssetID, AssetName, Status FROM ASSET WHERE Status = 'Unavailable';
+---------+----------------------+-------------+
| AssetID | AssetName            | Status      |
+---------+----------------------+-------------+
|     405 | Armoured Asset L-401 | Unavailable |
+---------+----------------------+-------------+
1 row in set (0.01 sec)
```

3. **Display high-criticality assets**

```
mysql> SELECT AssetID, AssetName, CriticalityLevel FROM ASSET WHERE CriticalityLevel = 'High';
+---------+----------------------+------------------+
| AssetID | AssetName            | CriticalityLevel |
+---------+----------------------+------------------+
|     401 | Fighter Asset A-101  | High             |
|     402 | Fighter Asset A-102  | High             |
|     404 | Naval Asset N-301    | High             |
|     405 | Armoured Asset L-401 | High             |
+---------+----------------------+------------------+
4 rows in set (0.00 sec)
```

4. **Display assets ordered by manufacturing year**

```
mysql> SELECT AssetName, MnfYear FROM ASSET ORDER BY MnfYear DESC;
+----------------------+---------+
| AssetName            | MnfYear |
+----------------------+---------+
| Naval Asset N-301    |    2024 |
| Fighter Asset A-101  |    2023 |
| Fighter Asset A-102  |    2022 |
| Armoured Asset L-401 |    2022 |
| Transport Asset T-201 |    2021 |
+----------------------+---------+
5 rows in set (0.00 sec)
```

5. **Display different asset statuses**

```
mysql> SELECT DISTINCT Status FROM ASSET;
+-------------+
| Status      |
+-------------+
| Degraded    |
| Ready       |
| Unavailable |
+-------------+
3 rows in set (0.01 sec)
```

7.5: AGGREGATE FUNCTIONS

1. **Count total assets**

```
mysql> SELECT COUNT(*) AS TotalAssets FROM ASSET;
+-------------+
| TotalAssets |
+-------------+
|           5 |
+-------------+
1 row in set (0.01 sec)
```

2. **Average maintenance cost**

```
mysql> SELECT AVG(CostIncurred) AS AverageMaintenanceCost FROM MAINTENANCE_EVENT;
+------------------------+
| AverageMaintenanceCost |
+------------------------+
|           55000.000000 |
+------------------------+
1 row in set (0.00 sec)
```

3. **Total maintenance cost**

```
mysql> SELECT SUM(CostIncurred) AS TotalMaintenanceCost FROM MAINTENANCE_EVENT;
+----------------------+
| TotalMaintenanceCost |
+----------------------+
|            110000.00 |
+----------------------+
1 row in set (0.00 sec)
```

4. **Maximum downtime**

```
mysql> SELECT MAX(DowntimeHours) AS MaximumDowntime FROM MAINTENANCE_EVENT;
+-----------------+
| MaximumDowntime |
+-----------------+
|            6.50 |
+-----------------+
1 row in set (0.00 sec)
```

5. **Minimum maintenance cost**

```
mysql> SELECT MIN(CostIncurred) AS MinimumMaintenanceCost FROM MAINTENANCE_EVENT;
+------------------------+
| MinimumMaintenanceCost |
+------------------------+
|               25000.00 |
+------------------------+
1 row in set (0.02 sec)
```

7.6: GROUP BY

1. **Number of assets by status**

```
mysql> SELECT Status, COUNT(*) AS AssetCount FROM ASSET GROUP BY Status;
+-------------+------------+
| Status      | AssetCount |
+-------------+------------+
| Degraded    |          2 |
| Ready       |          2 |
| Unavailable |          1 |
+-------------+------------+
3 rows in set (0.01 sec)
```

2. **Number of assets by criticality**

```
mysql> SELECT CriticalityLevel, COUNT(*) AS AssetCount FROM ASSET GROUP BY CriticalityLevel;
+------------------+------------+
| CriticalityLevel | AssetCount |
+------------------+------------+
| High             |          4 |
| Medium           |          1 |
+------------------+------------+
2 rows in set (0.00 sec)
```

3. **Total maintenance cost by maintenance type**

```
mysql> SELECT MaintenanceType,SUM(CostIncurred) AS TotalCost FROM MAINTENANCE_EVENT GROUP BY MaintenanceType;
+-----------------+-----------+
| MaintenanceType | TotalCost |
+-----------------+-----------+
| Inspection      |  25000.00 |
| Repair          |  85000.00 |
+-----------------+-----------+
2 rows in set (0.01 sec)
```

7.7: HAVING

1. **Asset statuses having more than one asset**

```
mysql> SELECT Status, COUNT(*) AS AssetCount FROM ASSET GROUP BY Status HAVING COUNT(*) > 1;
+----------+------------+
| Status   | AssetCount |
+----------+------------+
| Degraded |          2 |
| Ready    |          2 |
+----------+------------+
2 rows in set (0.00 sec)
```

2. **Maintenance types with total cost greater than 50000**

```
mysql> SELECT MaintenanceType,SUM(CostIncurred) AS TotalCost FROM MAINTENANCE_EVENT GROUP BY MaintenanceType HAVING SUM(CostIncurred) > 50000;
+-----------------+-----------+
| MaintenanceType | TotalCost |
+-----------------+-----------+
| Repair          |  85000.00 |
+-----------------+-----------+
1 row in set (0.01 sec)
```

7.8: JOINS

1. **Asset + Unit**

```
mysql> SELECT a.AssetName,u.UnitName,u.Location FROM ASSET a JOIN UNIT u ON a.UnitID = u.UnitID;
+-----------------------+---------------------------+------------+
| AssetName             | UnitName                  | Location   |
+-----------------------+---------------------------+------------+
| Fighter Asset A-101   | Fighter Squadron Alpha    | Bengaluru  |
| Fighter Asset A-102   | Fighter Squadron Alpha    | Bengaluru  |
| Transport Asset T-201 | Transport Squadron Bravo  | Chandigarh |
| Naval Asset N-301     | Naval Support Unit        | Kochi      |
| Armoured Asset L-401  | Armoured Unit Charlie     | Jaisalmer  |
+-----------------------+---------------------------+------------+
5 rows in set (0.02 sec)
```

2. **Asset + Unit + Service**

```
mysql> SELECT a.AssetName,u.UnitName,s.ServiceName FROM ASSET a JOIN UNIT u ON a.UnitID =u.UnitID JOIN SERVICE s ON u.ServiceID= s.ServiceID;
+-----------------------+---------------------------+-----------------+
| AssetName             | UnitName                  | ServiceName     |
+-----------------------+---------------------------+-----------------+
| Fighter Asset A-101   | Fighter Squadron Alpha    | Indian Air Force |
| Fighter Asset A-102   | Fighter Squadron Alpha    | Indian Air Force |
| Transport Asset T-201 | Transport Squadron Bravo  | Indian Air Force |
| Naval Asset N-301     | Naval Support Unit        | Indian Navy      |
| Armoured Asset L-401  | Armoured Unit Charlie     | Indian Army      |
+-----------------------+---------------------------+-----------------+
5 rows in set (0.01 sec)
```

3. **Asset + Subsystem**

```
mysql> SELECT a.AssetName,ss.SubsystemName,ss.CriticalityLevel FROM ASSET a JOIN SUBSYSTEM ss ON a.AssetID = ss.AssetID;
+-----------------------+-------------------+-------------------+
| AssetName             | SubsystemName     | CriticalityLevel |
+-----------------------+-------------------+-------------------+
| Fighter Asset A-101   | Engine System     | High              |
| Fighter Asset A-102   | Cooling System    | High              |
| Armoured Asset L-401  | Hydraulic System  | High              |
| Fighter Asset A-101   | Power System      | High              |
+-----------------------+-------------------+-------------------+
4 rows in set (0.01 sec)
```

4. **Subsystem + Component**

```
mysql> SELECT ss.SubsystemName,c.ComponentName,c.CriticalityLevel FROM SUBSYSTEM ss JOIN COMPONENT c ON ss.SubsystemID = c.SubsystemID;
+-----------------+--------------------+------------------+
| SubsystemName   | ComponentName      | CriticalityLevel |
+-----------------+--------------------+------------------+
| Engine System   | Fuel Pump          | High             |
| Engine System   | Engine Control Unit | High             |
| Cooling System  | Cooling Pump       | High             |
| Hydraulic System | Hydraulic Valve    | High             |
| Power System    | Power Controller   | High             |
+-----------------+--------------------+------------------+
5 rows in set (0.00 sec)
```

5. **Component + Spare Part**

```
mysql> SELECT c.ComponentName,sp.PartName,cs.QuantityRequired,cs.IsCritical
    -> FROM COMPONENT c JOIN COMPONENT_SPARE cs ON c.ComponentID = cs.ComponentID JOIN SPARE_PART sp ON cs.SparePartID = sp.SparePartID;
+-----------------+------------------------+------------------+------------+
| ComponentName   | PartName               | QuantityRequired | IsCritical |
+-----------------+------------------------+------------------+------------+
| Fuel Pump       | Fuel Pump Seal         |                2 | Yes        |
| Cooling Pump    | Cooling Pump Assembly  |                1 | Yes        |
| Hydraulic Valve | Hydraulic Valve Kit    |                1 | Yes        |
| Power Controller | Power Controller Module |                1 | Yes        |
+-----------------+------------------------+------------------+------------+
4 rows in set (0.01 sec)
```

6. **Spare Part + Supplier**

```
mysql> SELECT sp.PartName,su.SupplierName,ss.UnitPrice,ss.LeadTimeDay,ss.PreferredSupplier
    -> FROM SPARE_PART sp JOIN SUPPLIER_SPARE ss ON sp.SparePartID = ss.SparePartID JOIN SUPPLIER su ON ss.SupplierID = su.SupplierID;
+-------------------------+---------------------------+-----------+------------+-------------------+
| PartName                | SupplierName              | UnitPrice | LeadTimeDay | PreferredSupplier |
+-------------------------+---------------------------+-----------+------------+-------------------+
| Fuel Pump Seal          | Bharat Defence Components |   5000.00 |          7 | Yes               |
| Cooling Pump Assembly   | Aero Systems Manufacturing |  85000.00 |         14 | Yes               |
| Power Controller Module | Aero Systems Manufacturing | 120000.00 |         21 | Yes               |
+-------------------------+---------------------------+-----------+------------+-------------------+
3 rows in set (0.00 sec)
```

7. **Maintenance + User**

```
mysql> SELECT me.MaintenanceID,me.MaintenanceType,u.FullName AS PerformedBy,me.CostIncurred
    -> FROM MAINTENANCE_EVENT me JOIN `USER` u ON me.PerformedBy = u.UserID;
+---------------+-----------------+-------------+--------------+
| MaintenanceID | MaintenanceType | PerformedBy | CostIncurred |
+---------------+-----------------+-------------+--------------+
|           801 | Inspection      | Vikram Nair |     25000.00 |
|           802 | Repair          | Vikram Nair |     85000.00 |
+---------------+-----------------+-------------+--------------+
2 rows in set (0.01 sec)
```

7.9: MULTIPLE-TABLE JOIN

1. **Show complete asset hierarchy**

```
mysql> SELECT s.ServiceName,u.UnitName,a.AssetName,ss.SubsystemName,c.ComponentName
    -> FROM SERVICE s
    -> JOIN UNIT u ON s.ServiceID = u.ServiceID
    -> JOIN ASSET a ON u.UnitID = a.UnitID
    -> JOIN SUBSYSTEM ss ON a.AssetID = ss.AssetID
    -> JOIN COMPONENT c ON ss.SubsystemID = c.SubsystemID;
+-----------------+----------------------+---------------------+-----------------+--------------------+
| ServiceName     | UnitName             | AssetName           | SubsystemName   | ComponentName      |
+-----------------+----------------------+---------------------+-----------------+--------------------+
| Indian Air Force | Fighter Squadron Alpha | Fighter Asset A-101 | Engine System   | Fuel Pump          |
| Indian Air Force | Fighter Squadron Alpha | Fighter Asset A-101 | Engine System   | Engine Control Unit |
| Indian Air Force | Fighter Squadron Alpha | Fighter Asset A-101 | Power System    | Power Controller   |
| Indian Air Force | Fighter Squadron Alpha | Fighter Asset A-102 | Cooling System  | Cooling Pump       |
| Indian Army     | Armoured Unit Charlie  | Armoured Asset L-401 | Hydraulic System | Hydraulic Valve    |
+-----------------+----------------------+---------------------+-----------------+--------------------+
5 rows in set (0.00 sec)
```

7.10: SUBQUERIES

1. **Assets having above-average maintenance cost**

```
mysql> SELECT DISTINCT a.AssetID, a.AssetName
    -> FROM ASSET a
    -> JOIN MAINTENANCE_WORK_ORDER wo
    ->     ON a.AssetID = wo.AssetID
    -> JOIN MAINTENANCE_EVENT me
    ->     ON wo.WorkOrderID = me.WorkOrderID
    -> WHERE me.CostIncurred > (
    ->     SELECT AVG(CostIncurred)
    ->     FROM MAINTENANCE_EVENT
    -> );
+---------+---------------------+
| AssetID | AssetName           |
+---------+---------------------+
|     402 | Fighter Asset A-102 |
+---------+---------------------+
1 row in set (0.02 sec)
```

2. **Assets with at least one failure**

```
mysql> SELECT AssetID, AssetName FROM ASSET WHERE AssetID IN(SELECT AssetID FROM FAILURE_EVENT);
+---------+----------------------+
| AssetID | AssetName            |
+---------+----------------------+
|     401 | Fighter Asset A-101  |
|     402 | Fighter Asset A-102  |
|     405 | Armoured Asset L-401 |
+---------+----------------------+
3 rows in set (0.01 sec)
```

3. **Spare parts below reorder level**

```
mysql> SELECT SparePartID, PartName,
    ->        StockQuantity, ReorderLevel
    -> FROM SPARE_PART
    -> WHERE StockQuantity <= ReorderLevel;
Empty set (0.00 sec)
```

7.11: CORRELATED SUBQUERY

1. **Find spare parts whose stock is below their own reorder level.**

```
mysql> SELECT sp1.SparePartID,
    ->        sp1.PartName,
    ->        sp1.StockQuantity,
    ->        sp1.ReorderLevel
    -> FROM SPARE_PART sp1
    -> WHERE sp1.StockQuantity <= (
    ->     SELECT sp2.ReorderLevel
    ->     FROM SPARE_PART sp2
    ->     WHERE sp2.SparePartID = sp1.SparePartID
    -> );
Empty set (0.00 sec)
```

7.12: SET OPERATIONS

1. **Assets that are Ready or Degraded**

```
mysql> SELECT AssetName
    -> FROM ASSET
    -> WHERE Status = 'Ready'
    ->
    -> UNION
    ->
    -> SELECT AssetName
    -> FROM ASSET
    -> WHERE Status = 'Degraded';
+-----------------------+
| AssetName             |
+-----------------------+
| Transport Asset T-201 |
| Naval Asset N-301     |
| Fighter Asset A-101   |
| Fighter Asset A-102   |
+-----------------------+
4 rows in set (0.01 sec)
```

8. STORED PROCEDURES, FUNCTIONS, TRIGGERS AND TRANSACTIONS

8.1 Code:

```
/* ========================
   PART 1: VIEW
   ======================== */

-- View for asset readiness information.
CREATE OR REPLACE VIEW asset_readiness_view AS
SELECT
    a.AssetID,
    a.AssetName,
    a.Status,
    a.CriticalityLevel,
    u.UnitName,
    s.ServiceName
FROM ASSET a
JOIN UNIT u
    ON a.UnitID = u.UnitID
JOIN SERVICE s
    ON u.ServiceID = s.ServiceID;


-- Use the view
SELECT *
FROM asset_readiness_view;


/* ========================
   PART 2: INDEXES
   ======================== */


-- Frequently searched asset status
CREATE INDEX idx_asset_status
ON ASSET(Status);


-- Frequently searched component criticality
CREATE INDEX idx_component_criticality
ON COMPONENT(CriticalityLevel);


-- Failure lookup by asset
CREATE INDEX idx_failure_asset
ON FAILURE_EVENT(AssetID);
```

```sql
-- Maintenance lookup by work order
CREATE INDEX idx_maintenance_workorder
ON MAINTENANCE_EVENT(WorkOrderID);


/* ==============================
   PART 3: TRANSACTION
   ==============================*/


/*
   Example:
   Maintenance consumes spare parts.
   Both stock update and maintenance record
   should succeed together.
*/

START TRANSACTION;
UPDATE SPARE_PART
SET StockQuantity = StockQuantity - 2
WHERE SparePartID = 1101
AND StockQuantity >= 2;

COMMIT;


/*
   If something goes wrong:

   ROLLBACK;
*/


/* ============================
   PART 4: TRIGGER
   ==============================*/

--Trigger that automatically updates an asset’s status to Ready whenever a new maintenance
event of type Repair is inserted
DELIMITER $$

CREATE TRIGGER trg_asset_repair_status
AFTER INSERT ON maintenance_event
FOR EACH ROW
BEGIN
    IF NEW.MaintenanceType = 'Repair' THEN
        UPDATE asset
        SET Status = 'Ready'
        WHERE AssetID = (
            SELECT AssetID
```

```
            FROM maintenance_work_order
            WHERE WorkOrderID = NEW.WorkOrderID
        );
    END IF;
END$$

DELIMITER ;

--Testing
INSERT INTO maintenance_event
    (MaintenanceID, WorkOrderID, PerformedBy,
     MaintenanceType, Remarks)
VALUES
    (803, 701, 103, 'Repair',
     'Trigger testing');


/* ====================================
   PART 17: STORED FUNCTION
   ====================================*/

/*
   Returns the number of failures for an asset.
*/

DROP FUNCTION IF EXISTS get_failure_count;

DELIMITER $$

CREATE FUNCTION get_failure_count(
    p_asset_id INT
)
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE failure_count INT DEFAULT 0;

    SELECT COUNT(*)
    INTO failure_count
    FROM FAILURE_EVENT
    WHERE AssetID = p_asset_id;

    RETURN failure_count;
END$$

DELIMITER ;


-- Call the function
```

```sql
SELECT get_failure_count(401);


/* ====================================
   PART 5: STORED PROCEDURE
   ====================================*/

/*
   Updates the status of an asset.
*/
DROP PROCEDURE IF EXISTS update_asset_status;
DELIMITER $$
CREATE PROCEDURE update_asset_status(
    IN p_asset_id INT,
    IN p_status VARCHAR(50)
)
BEGIN
    UPDATE ASSET
    SET Status = p_status
    WHERE AssetID = p_asset_id;
END$$
DELIMITER ;


-- Call procedure
CALL update_asset_status(401, 'Ready');


/* ========================
   PART 6: CTE
   ========================*/


/*
   Calculate total maintenance cost
   for each work order.
*/

WITH MaintenanceSummary AS
(
    SELECT
        WorkOrderID,
        SUM(CostIncurred) AS TotalCost
    FROM MAINTENANCE_EVENT
    GROUP BY WorkOrderID
)
SELECT *
FROM MaintenanceSummary
ORDER BY TotalCost DESC;
```

```sql
/* ==================================
   PART 7: WINDOW FUNCTION
   ==================================*/


/*
   Rank maintenance events by cost.
*/

SELECT
    MaintenanceID,
    MaintenanceType,
    CostIncurred,
    RANK() OVER
    (
        ORDER BY CostIncurred DESC
    ) AS CostRank
FROM MAINTENANCE_EVENT;


/* ====================================
   PART 8: USEFUL PROJECT QUERY
   ====================================*/
--1. Find unavailable or degraded assets and their latest known failure.
SELECT
    a.AssetID,
    a.AssetName,
    a.Status,
    f.FailureType,
    f.Severity,
    f.OccurredAt
FROM ASSET a
JOIN FAILURE_EVENT f
    ON a.AssetID = f.AssetID
WHERE a.Status IN ('Unavailable', 'Degraded')
ORDER BY f.OccurredAt DESC;



-- 2.   Find critical components and the spare parts they require.
SELECT
    c.ComponentName,
    c.CriticalityLevel,
    sp.PartName,
    sp.StockQuantity,
    sp.ReorderLevel
FROM COMPONENT c
JOIN COMPONENT_SPARE cs
```

```sql
    ON c.ComponentID = cs.ComponentID
JOIN SPARE_PART sp
    ON cs.SparePartID = sp.SparePartID
WHERE c.CriticalityLevel = 'High';



-- 3.   Find components whose required spare parts are below reorder level.
SELECT
    c.ComponentName,
    sp.PartName,
    sp.StockQuantity,
    sp.ReorderLevel
FROM COMPONENT c
JOIN COMPONENT_SPARE cs
    ON c.ComponentID = cs.ComponentID
JOIN SPARE_PART sp
    ON cs.SparePartID = sp.SparePartID
WHERE sp.StockQuantity <= sp.ReorderLevel;



-- 4. Find component dependencies.
SELECT
    c1.ComponentName AS Component,
    c2.ComponentName AS DependsOn
FROM COMPONENT_DEPENDENCY cd
JOIN COMPONENT c1
    ON cd.ComponentID = c1.ComponentID
JOIN COMPONENT c2
    ON cd.DependsOnComponentID = c2.ComponentID;
```

Screenshot:

8.2: VIEW

1. View for asset readiness information.

```
mysql> CREATE OR REPLACE VIEW asset_readiness_view AS
    -> SELECT
    ->     a.AssetID,
    ->     a.AssetName,
    ->     a.Status,
    ->     a.CriticalityLevel,
    ->     u.UnitName,
    ->     s.ServiceName
    -> FROM ASSET a
    -> JOIN UNIT u
    ->     ON a.UnitID = u.UnitID
    -> JOIN SERVICE s
    ->     ON u.ServiceID = s.ServiceID;
Query OK, 0 rows affected (0.07 sec)

mysql> SELECT *
    -> FROM asset_readiness_view;
+---------+----------------------+-------------+-----------------+--------------------------+-----------------+
| AssetID | AssetName            | Status      | CriticalityLevel | UnitName                 | ServiceName     |
+---------+----------------------+-------------+-----------------+--------------------------+-----------------+
|     401 | Fighter Asset A-101  | Degraded    | High            | Fighter Squadron Alpha   | Indian Air Force |
|     402 | Fighter Asset A-102  | Degraded    | High            | Fighter Squadron Alpha   | Indian Air Force |
|     403 | Transport Asset T-201 | Ready      | Medium          | Transport Squadron Bravo | Indian Air Force |
|     404 | Naval Asset N-301    | Ready       | High            | Naval Support Unit       | Indian Navy      |
|     405 | Armoured Asset L-401 | Unavailable | High            | Armoured Unit Charlie    | Indian Army      |
+---------+----------------------+-------------+-----------------+--------------------------+-----------------+
5 rows in set (0.01 sec)
```

8.3: INDEXES

1. **Frequently searched asset status**

```
mysql> CREATE INDEX idx_asset_status ON ASSET(Status);
Query OK, 0 rows affected (0.26 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

2. **Frequently searched component criticality**

```
mysql> CREATE INDEX idx_component_criticality ON COMPONENT(CriticalityLevel);
Query OK, 0 rows affected (0.11 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

3. **Failure lookup by asset**

```
mysql> CREATE INDEX idx_failure_asset ON FAILURE_EVENT(AssetID);
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

4. **Maintenance lookup by work order**

```
mysql> CREATE INDEX idx_maintenance_workorder ON MAINTENANCE_EVENT(WorkOrderID);
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

8.4: TRANSACTION

1. **Example: Maintenance consumes spare parts. Both stock update and maintenance record should succeed together.**

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE SPARE_PART
    -> SET StockQuantity = StockQuantity - 2
    -> WHERE SparePartID = 1101
    ->   AND StockQuantity >= 2;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

2. **If something goes wrong:**

```
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

8.5: TRIGGER

1. **Trigger that automatically updates an asset's status to `Ready` whenever a new maintenance event of type `Repair` is inserted**

```
mysql> CREATE TRIGGER trg_asset_repair_status
    -> AFTER INSERT ON maintenance_event
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.MaintenanceType = 'Repair' THEN
    ->         UPDATE asset
    ->         SET Status = 'Ready'
    ->         WHERE AssetID = (
    ->             SELECT AssetID
    ->             FROM maintenance_work_order
    ->             WHERE WorkOrderID = NEW.WorkOrderID
    ->         );
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.04 sec)
```

Testing the trigger

```
mysql> SELECT AssetID, AssetName, Status
    -> FROM asset
    -> WHERE AssetID = 402;
+---------+---------------------+----------+
| AssetID | AssetName           | Status   |
+---------+---------------------+----------+
|     402 | Fighter Asset A-102 | Degraded |
+---------+---------------------+----------+
1 row in set (0.00 sec)

mysql> INSERT INTO maintenance_event
    ->     (MaintenanceID, WorkOrderID, PerformedBy,
    ->      MaintenanceType, Remarks)
    -> VALUES
    ->     (803, 701, 103, 'Repair',
    ->      'Trigger testing');
Query OK, 1 row affected (0.02 sec)

mysql> SELECT AssetID, AssetName, Status
    -> FROM asset
    -> WHERE AssetID = 402;
+---------+---------------------+--------+
| AssetID | AssetName           | Status |
+---------+---------------------+--------+
|     402 | Fighter Asset A-102 | Ready  |
+---------+---------------------+--------+
1 row in set (0.00 sec)
```

8.6: STORED FUNCTION

1. **Returns the number of failures for an asset.**

```
mysql> DROP FUNCTION IF EXISTS get_failure_count;
Query OK, 0 rows affected, 1 warning (0.08 sec)

mysql>
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION get_failure_count(
    ->     p_asset_id INT
    -> )
    -> RETURNS INT
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE failure_count INT DEFAULT 0;
    ->
    ->     SELECT COUNT(*)
    ->     INTO failure_count
    ->     FROM FAILURE_EVENT
    ->     WHERE AssetID = p_asset_id;
    ->
    ->     RETURN failure_count;
    -> END$$
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql>
```

2. **Call the function**

```
mysql> SELECT get_failure_count(401);
+------------------------+
| get_failure_count(401) |
+------------------------+
|                      1 |
+------------------------+
1 row in set (0.01 sec)
```

8.7: STORED PROCEDURE

1. **Updates the status of an asset.**

```
mysql> DROP PROCEDURE IF EXISTS update_asset_status;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql>
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE update_asset_status(
    ->     IN p_asset_id INT,
    ->     IN p_status VARCHAR(50)
    -> )
    -> BEGIN
    ->     UPDATE ASSET
    ->     SET Status = p_status
    ->     WHERE AssetID = p_asset_id;
    -> END$$
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql>
```

```
mysql> CALL update_asset_status(401, 'Ready');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT AssetID, AssetName, Status
    -> FROM ASSET
    -> WHERE AssetID = 401;
+---------+---------------------+--------+
| AssetID | AssetName           | Status |
+---------+---------------------+--------+
|     401 | Fighter Asset A-101 | Ready  |
+---------+---------------------+--------+
1 row in set (0.00 sec)
```

8.8: CTE

1. **Calculate total maintenance cost for each work order.**

```
mysql> WITH MaintenanceSummary AS (
    ->     SELECT
    ->         WorkOrderID,
    ->         SUM(CostIncurred) AS TotalCost
    ->     FROM MAINTENANCE_EVENT
    ->     GROUP BY WorkOrderID
    -> )
    -> SELECT *
    -> FROM MaintenanceSummary
    -> ORDER BY TotalCost DESC;
+-------------+-----------+
| WorkOrderID | TotalCost |
+-------------+-----------+
|         701 |  85000.00 |
|         702 |  25000.00 |
+-------------+-----------+
2 rows in set (0.00 sec)
```

8.9: WINDOW FUNCTION

1. **Rank maintenance events by cost.**

```
mysql> SELECT
    ->     MaintenanceID,
    ->     MaintenanceType,
    ->     CostIncurred,
    ->     RANK() OVER (
    ->         ORDER BY CostIncurred DESC
    ->     ) AS CostRank
    -> FROM MAINTENANCE_EVENT;
+---------------+-----------------+--------------+----------+
| MaintenanceID | MaintenanceType | CostIncurred | CostRank |
+---------------+-----------------+--------------+----------+
|           802 | Repair          |     85000.00 |        1 |
|           801 | Inspection      |     25000.00 |        2 |
|           803 | Repair          |         NULL |        3 |
+---------------+-----------------+--------------+----------+
3 rows in set (0.00 sec)
```

8.10: USEFUL PROJECT QUERY

1. **Find unavailable or degraded assets and their latest known failure.**

```
mysql> SELECT
    ->     a.AssetID,
    ->     a.AssetName,
    ->     a.Status,
    ->     f.FailureType,
    ->     f.Severity,
    ->     f.OccurredAt
    -> FROM ASSET a
    -> JOIN FAILURE_EVENT f
    ->     ON a.AssetID = f.AssetID
    -> WHERE a.Status IN ('Unavailable', 'Degraded')
    -> ORDER BY f.OccurredAt DESC;
+---------+---------------------+-------------+------------------+----------+---------------------+
| AssetID | AssetName           | Status      | FailureType      | Severity | OccurredAt          |
+---------+---------------------+-------------+------------------+----------+---------------------+
|     405 | Armoured Asset L-401 | Unavailable | Hydraulic Failure | Medium   | 2026-09-21 14:15:00 |
+---------+---------------------+-------------+------------------+----------+---------------------+
1 row in set (0.00 sec)
```

2. **Find critical components and the spare parts they require.**

```
mysql> SELECT
    ->     c.ComponentName,
    ->     c.CriticalityLevel,
    ->     sp.PartName,
    ->     sp.StockQuantity,
    ->     sp.ReorderLevel
    -> FROM COMPONENT c
    -> JOIN COMPONENT_SPARE cs
    ->     ON c.ComponentID = cs.ComponentID
    -> JOIN SPARE_PART sp
    ->     ON cs.SparePartID = sp.SparePartID
    -> WHERE c.CriticalityLevel = 'High';
+-----------------+-----------------+-------------------------+--------------+--------------+
| ComponentName   | CriticalityLevel | PartName                | StockQuantity | ReorderLevel |
+-----------------+-----------------+-------------------------+--------------+--------------+
| Fuel Pump       | High            | Fuel Pump Seal          |           48 |           10 |
| Cooling Pump    | High            | Cooling Pump Assembly   |            8 |            3 |
| Hydraulic Valve | High            | Hydraulic Valve Kit     |           15 |            5 |
| Power Controller | High           | Power Controller Module |            6 |            2 |
+-----------------+-----------------+-------------------------+--------------+--------------+
4 rows in set (0.01 sec)
```

3. **Find components whose required spare parts are below reorder level.**

```
mysql> SELECT
    ->     c.ComponentName,
    ->     sp.PartName,
    ->     sp.StockQuantity,
    ->     sp.ReorderLevel
    -> FROM COMPONENT c
    -> JOIN COMPONENT_SPARE cs
    ->     ON c.ComponentID = cs.ComponentID
    -> JOIN SPARE_PART sp
    ->     ON cs.SparePartID = sp.SparePartID
    -> WHERE sp.StockQuantity <= sp.ReorderLevel;
Empty set (0.00 sec)
```

4. **Find component dependencies.**

```
mysql> SELECT
    ->     c1.ComponentName AS Component,
    ->     c2.ComponentName AS DependsOn
    -> FROM COMPONENT_DEPENDENCY cd
    -> JOIN COMPONENT c1
    ->     ON cd.ComponentID = c1.ComponentID
    -> JOIN COMPONENT c2
    ->     ON cd.DependsOnComponentID = c2.ComponentID;
+--------------------+-----------+
| Component          | DependsOn |
+--------------------+-----------+
| Engine Control Unit | Fuel Pump |
| Cooling Pump       | Fuel Pump |
+--------------------+-----------+
2 rows in set (0.01 sec)
```

9. FRONT END DEVELOPMENT

9.1 Overview

The front end of the Defence Asset Readiness and Maintenance Dependency System is developed using Next.js with the React framework. It provides a web-based interface for authorized users to access and manage defence asset information.

The application includes pages for the dashboard, assets, maintenance work orders, component dependencies, inventory, and reports. The dashboard presents asset-readiness information and maintenance summaries using data retrieved from the backend.

The front end communicates with the application layer through API requests. The API handlers process the requests, interact with the MySQL database, and return responses in JSON format. The interface displays the returned data and provides forms and controls for supported operations.

9.2 Database Used by the Application

The application uses MySQL as its relational database. The database stores information about countries, services, units, asset types, assets, components, component dependencies, spare parts, suppliers, maintenance work orders, maintenance events, inspections, failures, and users.

The database schema uses primary keys to identify records and foreign keys to maintain relationships between related tables. For example, an asset is associated with a unit and an asset type, while a maintenance work order is associated with an asset. Component dependencies are represented using the COMPONENT_DEPENDENCY table.

The application accesses MySQL through backend API handlers using prepared statements. This keeps database operations on the server side and avoids directly exposing database credentials to the browser. Redis is used as a cache for dashboard summary data. The cache has a short time-to-live, while MySQL remains the system of record for application data.

9.3 Authentication and Sessions

The application uses session-based authentication to restrict access to protected pages and API routes. Users submit their login credentials through the login interface. The backend verifies the credentials and, when authentication succeeds, creates a session.

The session is maintained using an HTTP-only cookie, which is not accessible to client-side JavaScript. Middleware checks the session cookie when a user requests a protected route. Unauthenticated requests are redirected to the login page or rejected by the relevant API handler.

This approach helps protect application pages and operations from unauthorized access. Authentication and authorization checks are handled by the backend rather than relying only on the visibility of front-end controls.

9.4 API Endpoints

The application uses Next.js API Route Handlers to process requests from the front end. The handlers validate incoming data, perform the required database operations, and return JSON responses. Protected endpoints require a valid authenticated session.

Table 9.1: Main API endpoint groups

Method	Endpoint	Purpose
GET / POST	/api/auth/login	Authenticate a user and establish a session.
POST	/api/auth/logout	End the current user session.
GET	/api/assets	Retrieve asset records for the application.
POST	/api/assets	Register an asset.
GET / PATCH	/api/assets/[id]	Retrieve or update a particular asset.
GET / POST	/api/work-orders	Retrieve or create maintenance work orders.
GET / POST	/api/dependencies	Retrieve or manage component dependency records, where supported by the existing route.
GET	/api/health/redis	Check Redis cache configuration and connectivity.

The endpoint names and supported methods should be verified against the implemented API routes before final submission.

9.5 Dashboard Caching Using Redis

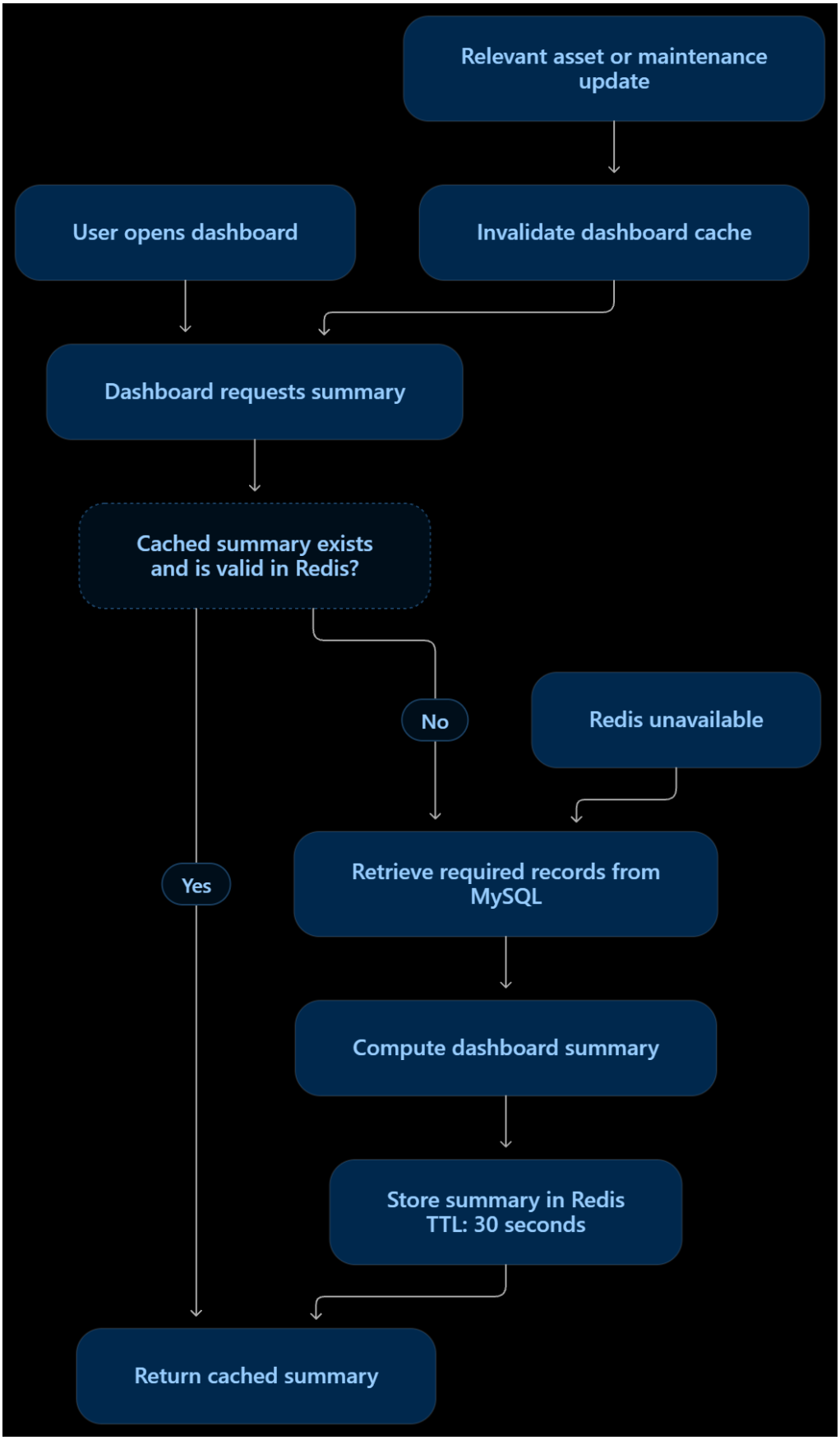
MySQL is the primary relational database and stores the application's operational records. Redis is used as a cache for dashboard summary data to reduce repeated database queries when the dashboard is requested.

When the dashboard summary is requested, the application checks whether a cached result is available in Redis. If the cache contains a valid result, the application can return it without recomputing the summary. If the cache is missing or expired, the application retrieves the required information from MySQL and stores the result in Redis for subsequent requests.

The dashboard cache uses a **30-second time-to-live (TTL)**. Relevant data changes trigger cache invalidation so that later dashboard requests can rebuild the summary using updated database records. If Redis is unavailable, the application falls back to retrieving the data from MySQL.

Redis therefore improves dashboard response efficiency but does not replace MySQL as the source of truth.

Fig. 9.1: Flow of dashboard summary caching using Redis


Relevant asset or maintenance update
User opens dashboard
Invalidate dashboard cache
Dashboard requests summary
Cached summary exists and is valid in Redis?
No
Redis unavailable
Yes
Retrieve required records from MySQL
Compute dashboard summary
Store summary in Redis TTL: 30 seconds
Return cached summary

9.7 User Interface Screens

The application provides a web-based interface for managing defence assets and their maintenance activities. The interface is organized into pages that allow authorized users to view asset readiness, manage asset records, track maintenance work orders, review component dependencies, and monitor spare-part inventory.

Fig. 9.2: Login page

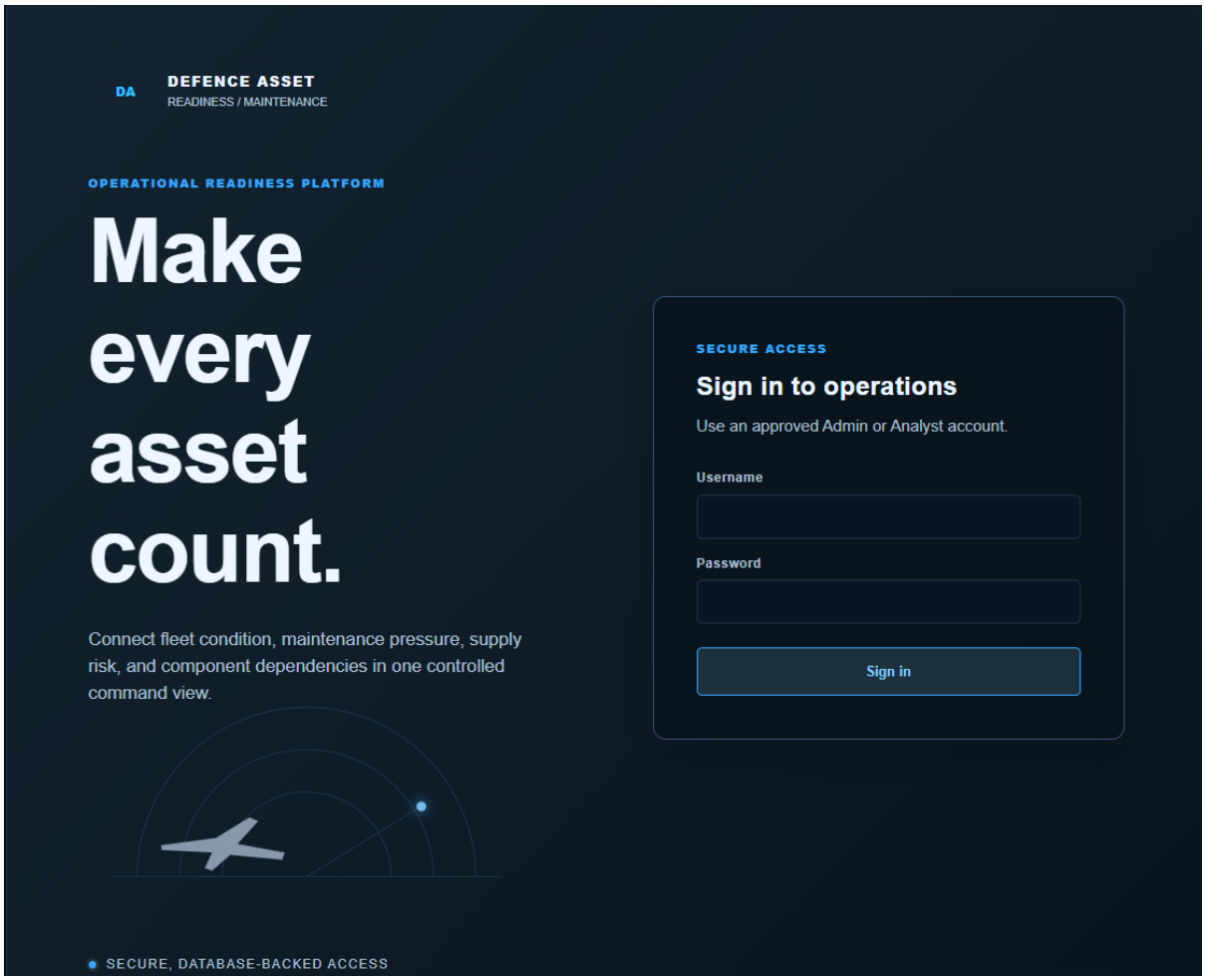


Fig. 9.3: Dashboard

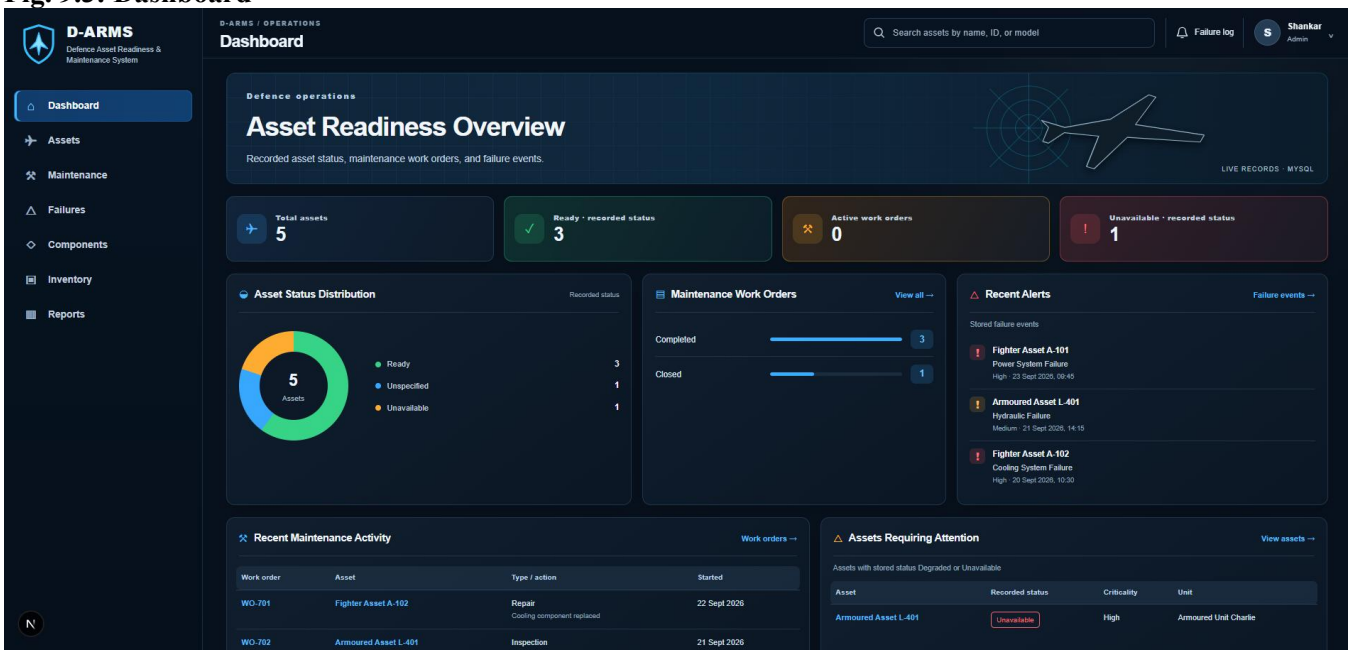
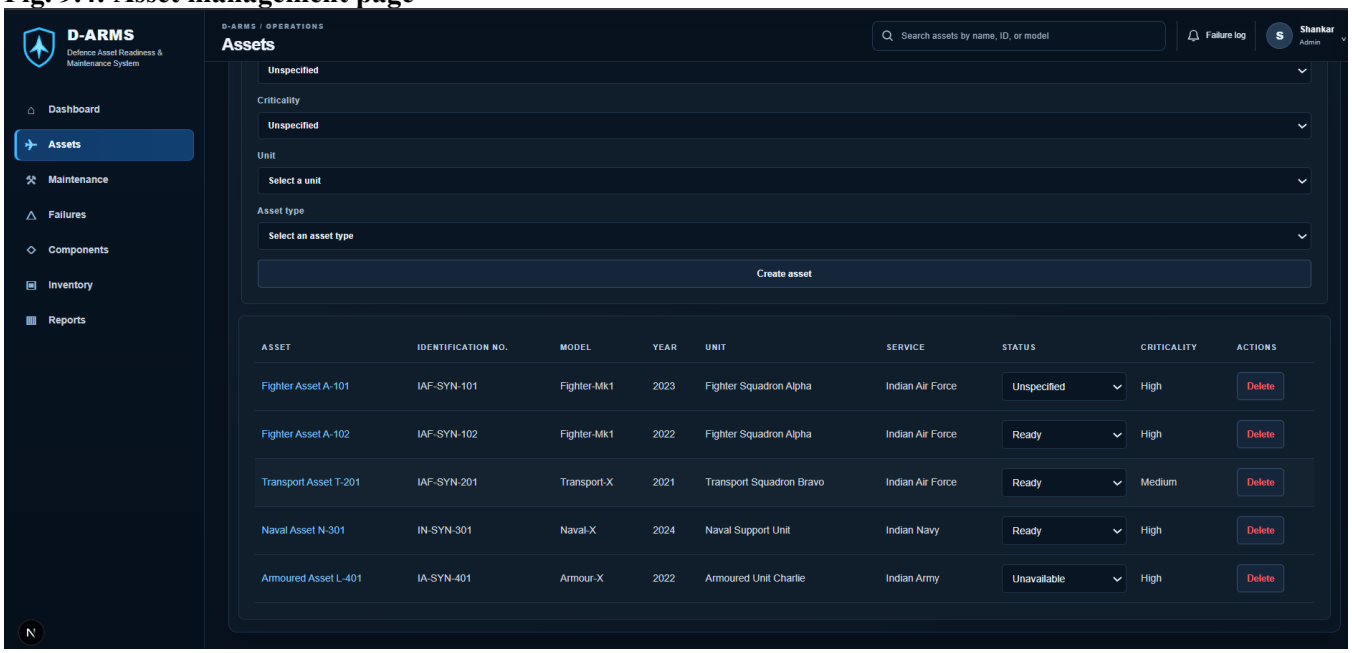
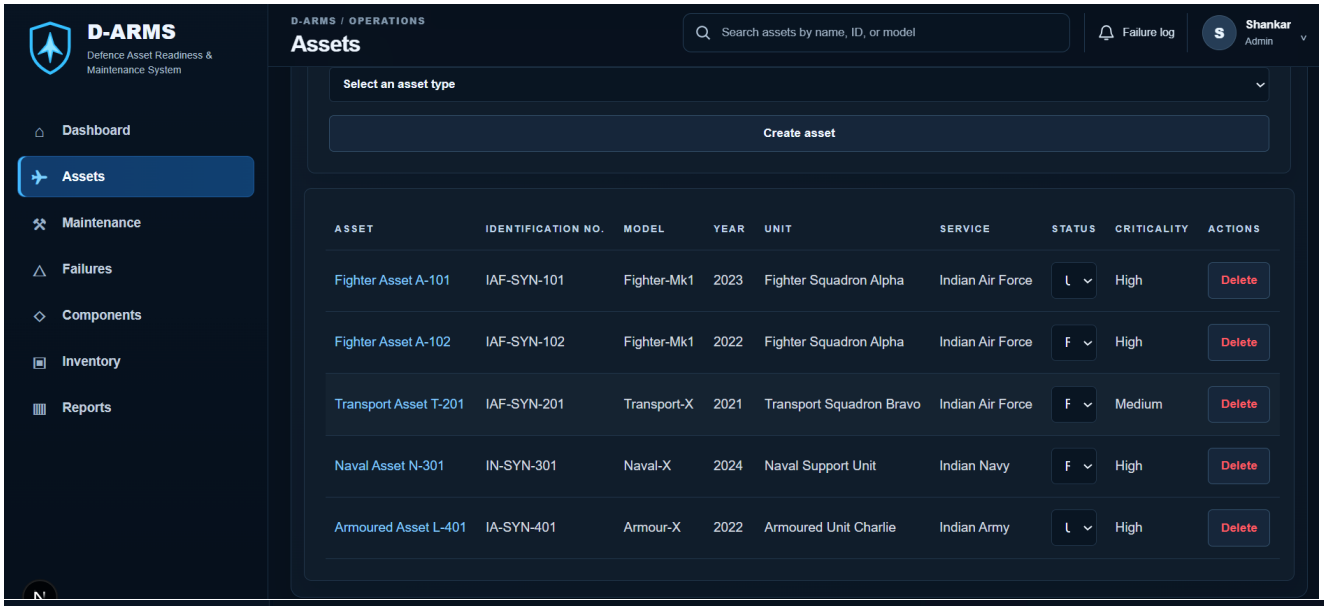


Fig. 9.4: Asset management page




D-ARMS
Defence Asset Readiness & Maintenance System
D-ARMS / OPERATIONS
Assets
Search assets by name, ID, or model
Failure log
Shankar
Admin
Dashboard
Assets
Maintenance
Failures
Components
Inventory
Reports
Select an asset type
Create asset
ASSET
IDENTIFICATION NO.
MODEL
YEAR
UNIT
SERVICE
STATUS
CRITICALITY
ACTIONS
Fighter Asset A-101
IAF-SYN-101
Fighter-Mk1
2023
Fighter Squadron Alpha
Indian Air Force
High
Delete
Fighter Asset A-102
IAF-SYN-102
Fighter-Mk1
2022
Fighter Squadron Alpha
Indian Air Force
High
Delete
Transport Asset T-201
IAF-SYN-201
Transport-X
2021
Transport Squadron Bravo
Indian Air Force
Medium
Delete
Naval Asset N-301
IN-SYN-301
Naval-X
2024
Naval Support Unit
Indian Navy
High
Delete
Armoured Asset L-401
IA-SYN-401
Armour-X
2022
Armoured Unit Charlie
Indian Army
High
Delete


D-ARMS
Defence Asset Readiness & Maintenance System
D-ARMS / OPERATIONS
Assets
Search assets by name, ID, or model
Failure log
Shankar
Admin
Dashboard
Assets
Maintenance
Failures
Components
Inventory
Reports
FLEET DOSSIER / ASSET DETAIL
FIGHTER ASSET A-101
Fighter Squadron Alpha · Indian Air Force · Fighter-Mk1
← Back to assets
Recorded asset status
DA
Unspecified
Identification no.
IAF-SYN-101
Manufacture year
2023
Criticality
High
Unit
Fighter Squadron Alpha
Service
Indian Air Force
Recorded failures
Power System Failure
High · 23 Sept 2026, 09:45
Components

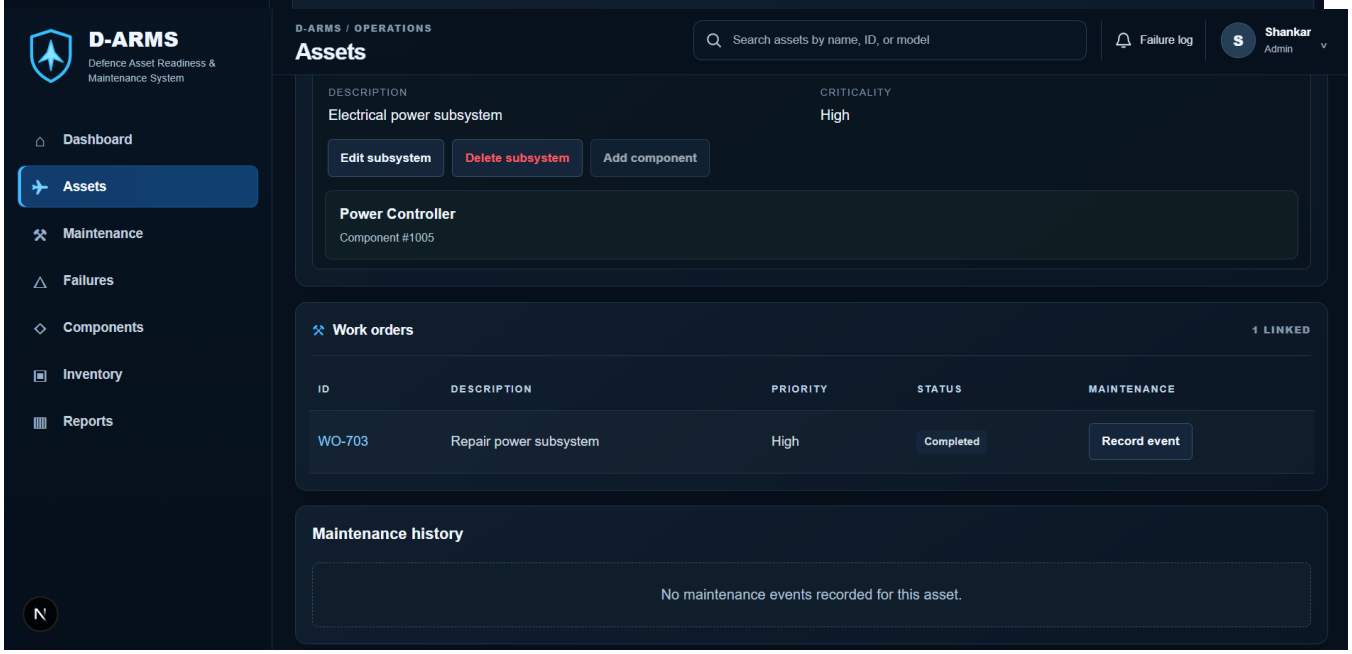

D-ARMS
Defence Asset Readiness & Maintenance System
D-ARMS / OPERATIONS
Assets
Search assets by name, ID, or model
Failure log
Shankar
Admin
Dashboard
Assets
Maintenance
Failures
Components
Inventory
Reports
DESCRIPTION
Electrical power subsystem
CRITICALITY
High
Edit subsystem
Delete subsystem
Add component
Power Controller
Component #1005
Work orders
1 LINKED
ID
DESCRIPTION
PRIORITY
STATUS
MAINTENANCE
WO-703
Repair power subsystem
High
Completed
Record event
Maintenance history
No maintenance events recorded for this asset.

Fig. 9.5: Maintenance work orders page

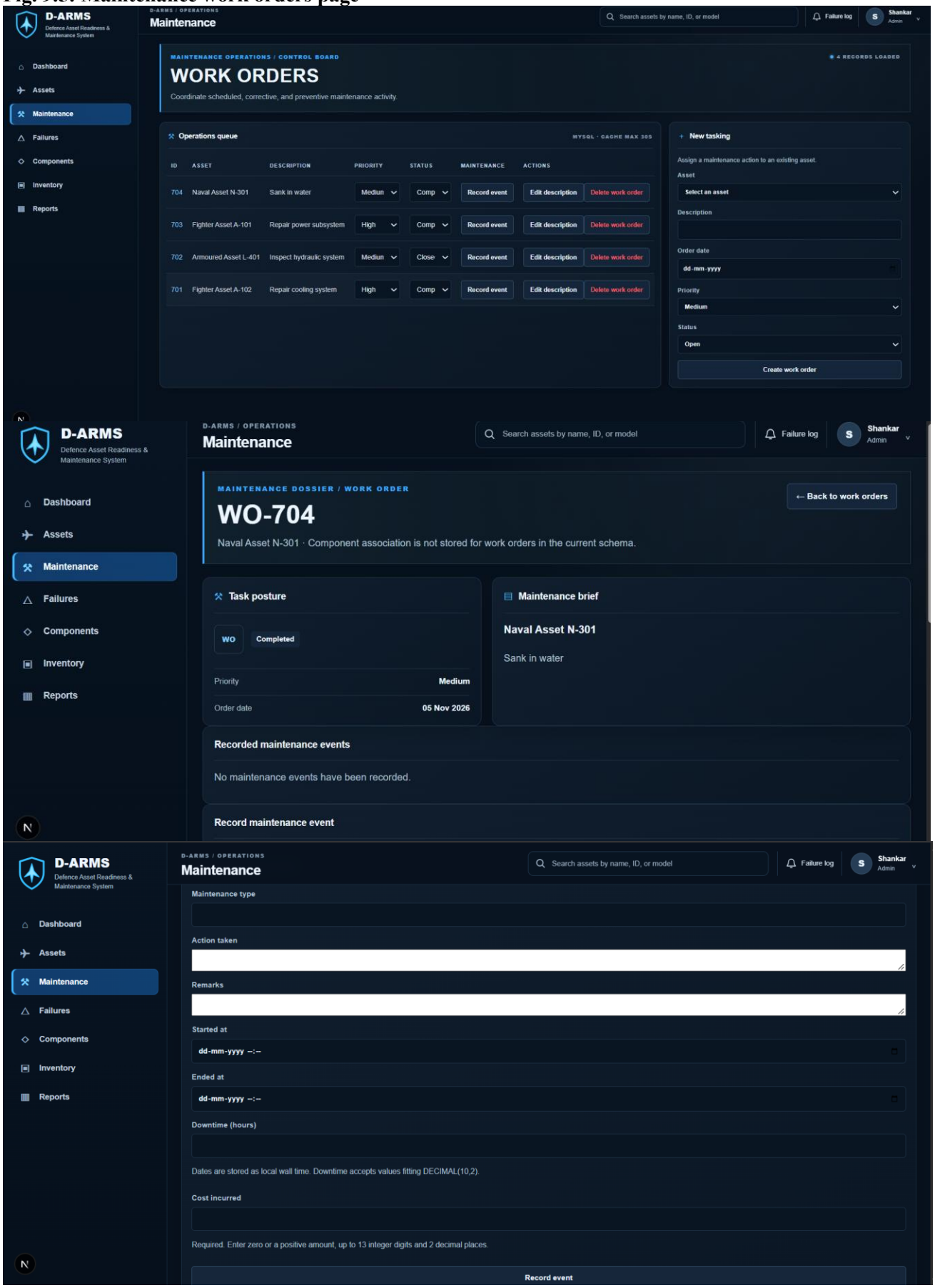


Fig. 9.6: Failure events Page

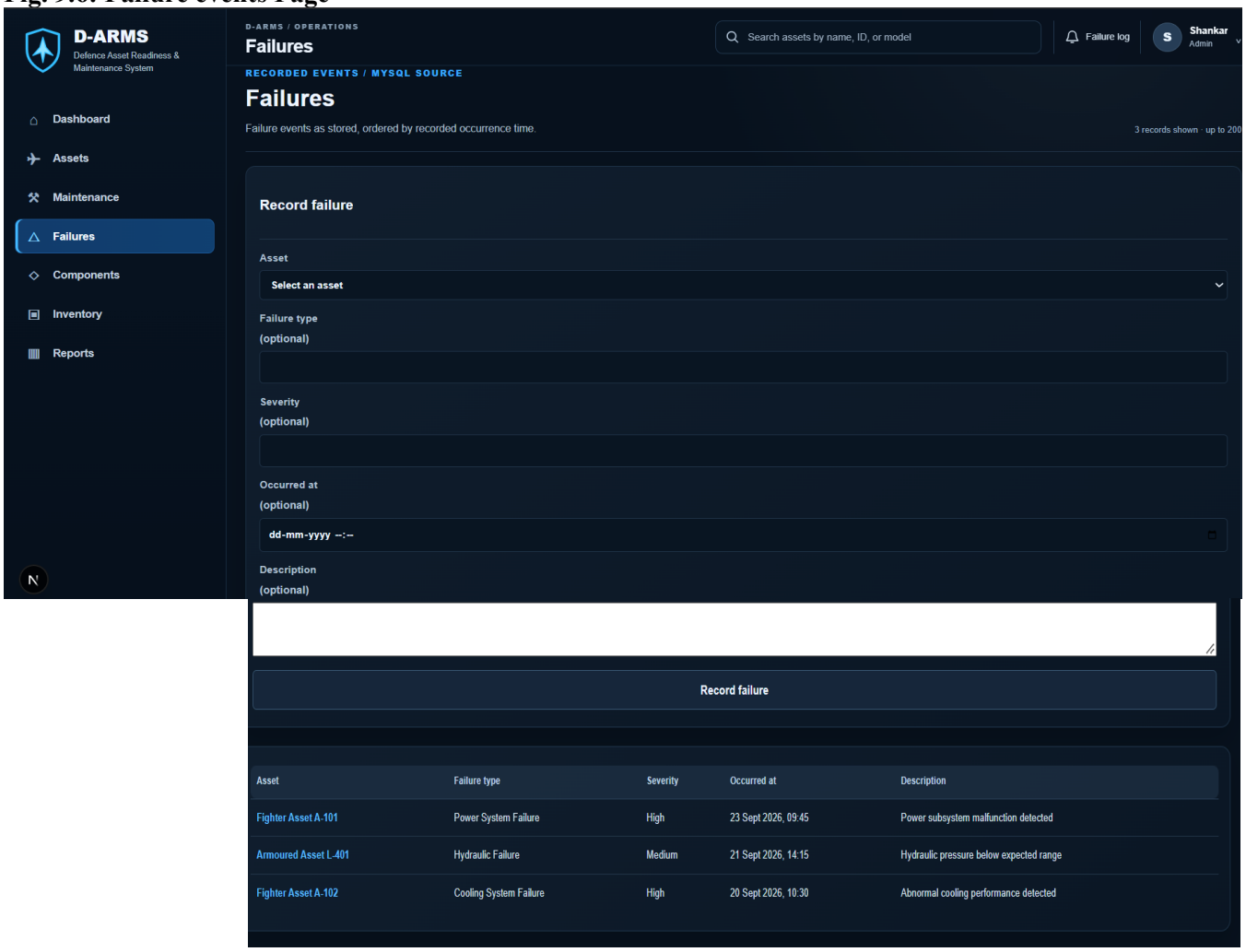
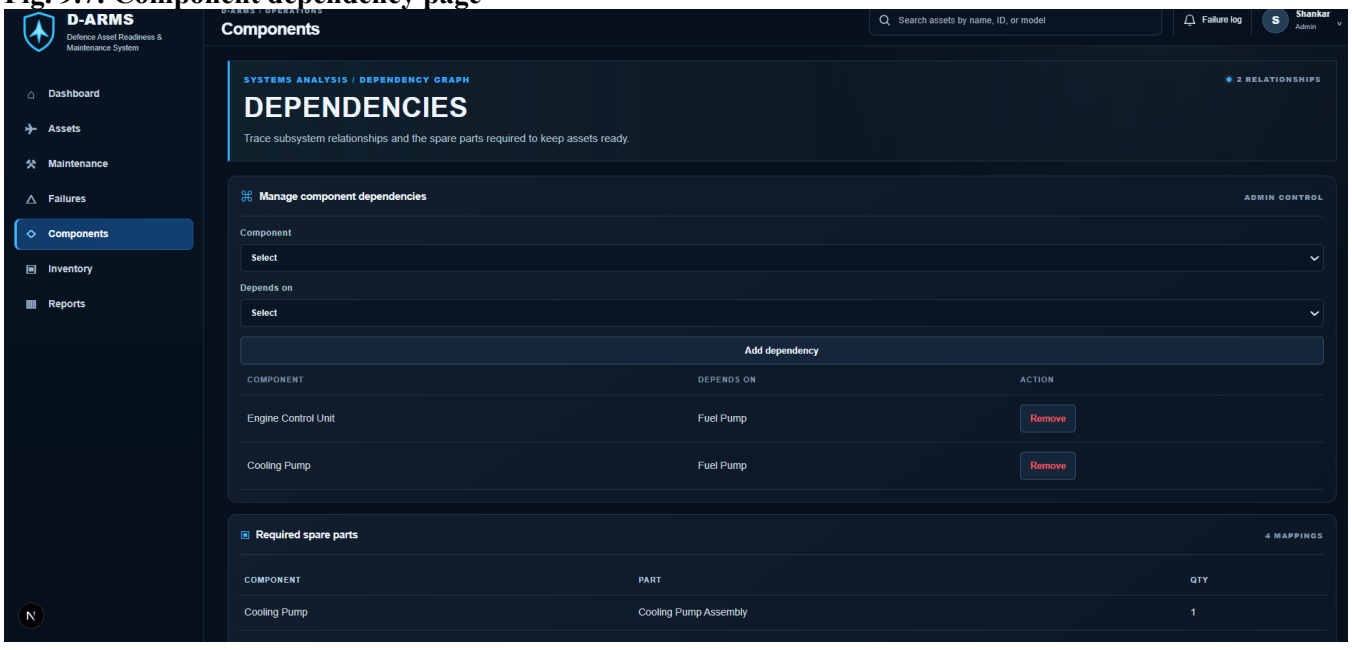


Fig. 9.7: Component dependency page



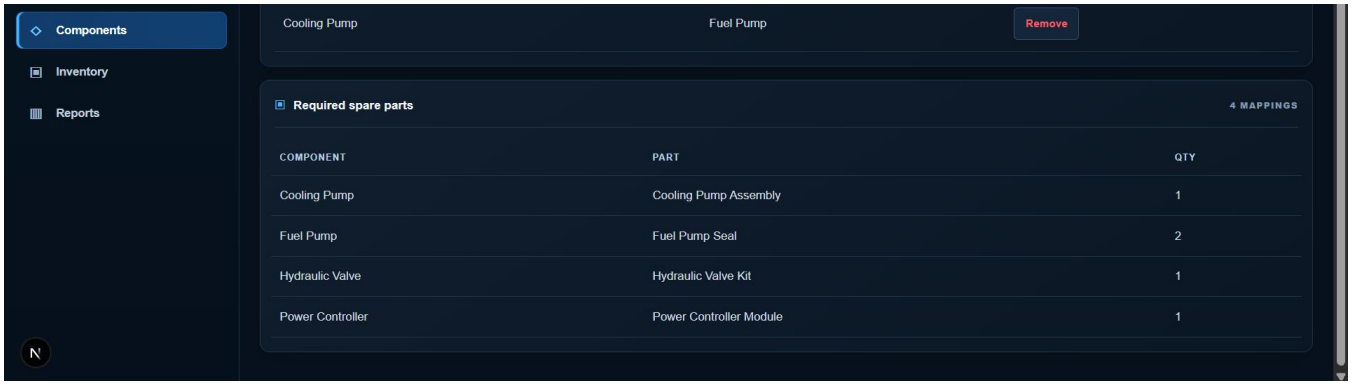


Fig. 9.8: Inventory page

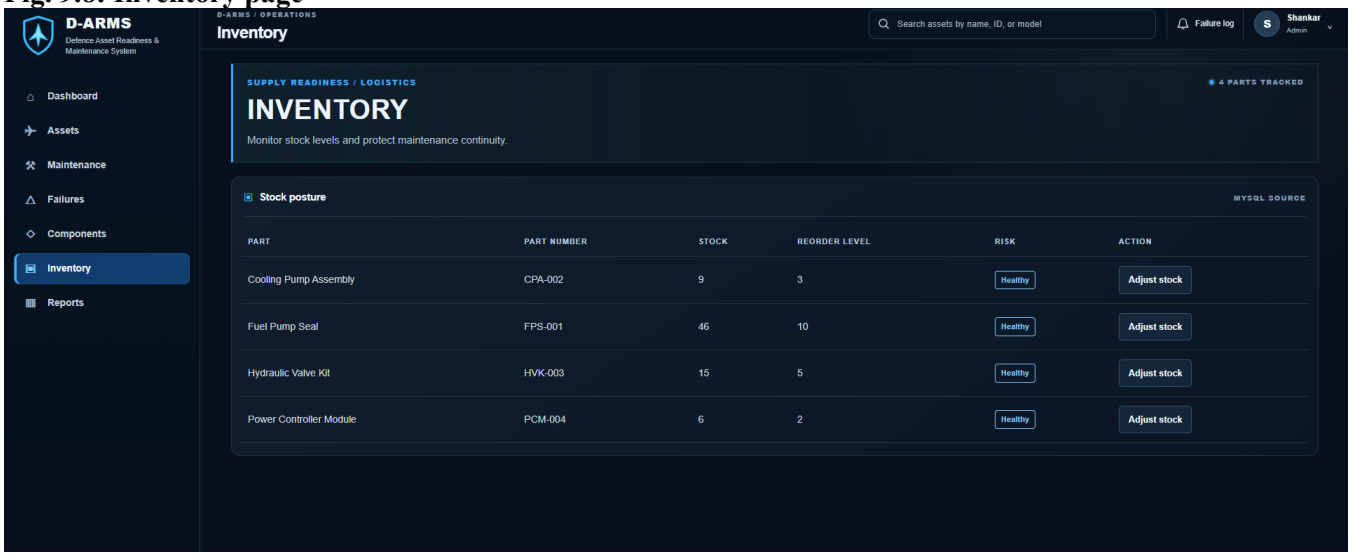


Fig. 9.9: Reports page

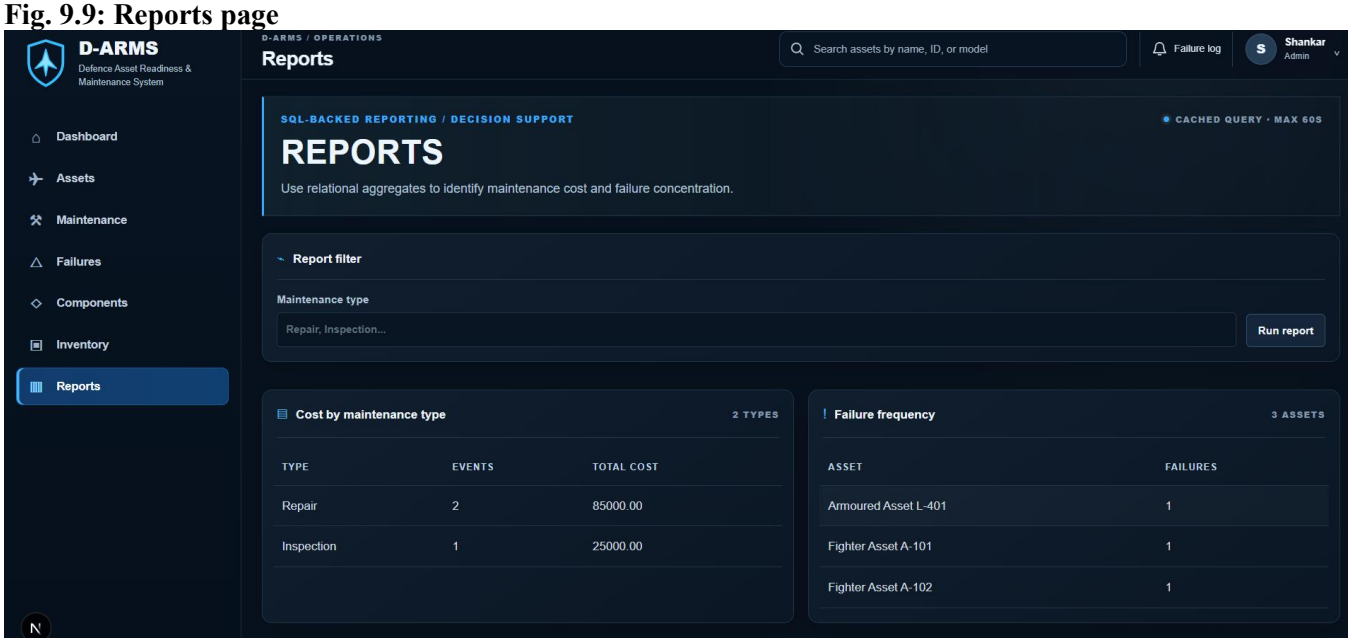
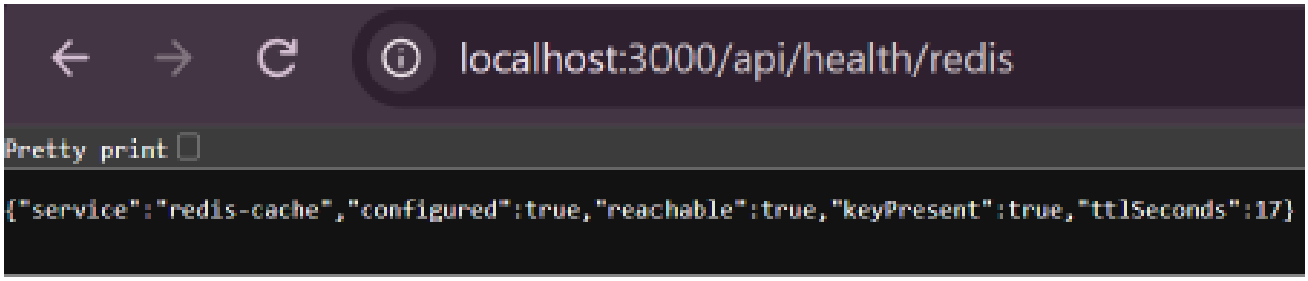


Fig. 9.10: Redis Cache Health Status




```
raksh@Raksh:~$ redis-cli -h 127.0.0.1 -p 6379 GET 'darms:v1:81704283497d:assets-cbbc43de-9e12-4b9d-8e5d-91542df8c12a:fai
lures-a3341d1b-03dc-4d4d-a8af-17db302a30c3:maintenance-events-5be880b2-9471-4e3a-9282-f2288ddc1a20:work-orders-d21f77f2-
013b-4d9a-ab55-eef128e397f2:4fe1bdd0c191a418b1c03514034d3043'
"{\"version\":1,\"data\":{\"version\":2,\"totalAssets\":5,\"assetStatuses\":[{\"Status\":\"Ready\",\"count\":3},{\"Statu
s\":null,\"count\":1},{\"Status\":\"Unavailable\",\"count\":1}],\"workOrderStatuses\":[{\"Status\":\"Completed\",\"count
\":3},{\"Status\":\"Closed\",\"count\":1}],\"recentFailures\":[{\"FailureID\":503,\"AssetID\":401,\"AssetName\":\"Fighte
r Asset A-101\",\"FailureType\":\"Power System Failure\",\"Severity\":\"High\",\"OccurredAt\":\"2026-09-23T04:15:00.000Z
\"},{\"FailureID\":502,\"AssetID\":405,\"AssetName\":\"Armoured Asset L-401\",\"FailureType\":\"Hydraulic Failure\",\"Se
verity\":\"Medium\",\"OccurredAt\":\"2026-09-21T08:45:00.000Z\"},{\"FailureID\":501,\"AssetID\":402,\"AssetName\":\"Figh
ter Asset A-102\",\"FailureType\":\"Cooling System Failure\",\"Severity\":\"High\",\"OccurredAt\":\"2026-09-20T05:00:00.
000Z\"}],\"recentMaintenance\":[{\"MaintenanceID\":802,\"WorkOrderID\":701,\"AssetID\":402,\"AssetName\":\"Fighter Asset
 A-102\",\"MaintenanceType\":\"Repair\",\"ActionTaken\":\"Cooling component replaced\",\"StartDateTime\":\"2026-09-22T03
:30:00.000Z\"},{\"MaintenanceID\":801,\"WorkOrderID\":702,\"AssetID\":405,\"AssetName\":\"Armoured Asset L-401\",\"Maint
enanceType\":\"Inspection\",\"ActionTaken\":\"Hydraulic system inspected\",\"StartDateTime\":\"2026-09-21T10:30:00.000Z\
"},{\"MaintenanceID\":803,\"WorkOrderID\":701,\"AssetID\":402,\"AssetName\":\"Fighter Asset A-102\",\"MaintenanceType\":
\"Repair\",\"ActionTaken\":null,\"StartDateTime\":null}],\"attentionAssets\":[{\"AssetID\":405,\"AssetName\":\"Armoured
Asset L-401\",\"Status\":\"Unavailable\",\"CriticalityLevel\":\"High\",\"UnitName\":\"Armoured Unit Charlie\"}]}}"
raksh@Raksh:~$ redis-cli -h 127.0.0.1 -p 6379 TTL 'darms:v1:81704283497d:assets-cbbc43de-9e12-4b9d-8e5d-91542df8c12a:fai
lures-a3341d1b-03dc-4d4d-a8af-17db302a30c3:maintenance-events-5be880b2-9471-4e3a-9282-f2288ddc1a20:work-orders-d21f77f2-
013b-4d9a-ab55-eef128e397f2:4fe1bdd0c191a418b1c03514034d3043'
(integer) 27
raksh@Raksh:~$
```

REFERENCES

[1] [1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

[2] [2] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2020.

[3] [3] Oracle Corporation, "MySQL 8.0 Reference Manual: SQL Statements, Stored Programs, Triggers, Views and Transactions." https://dev.mysql.com/doc/refman/8.0/en/

[4] [4] Vercel, "Next.js Documentation: App Router, Route Handlers and Authentication." https://nextjs.org/docs

[5] [5] Meta, "React Documentation." https://react.dev

[6] [6] Oracle Corporation, "MySQL2: MySQL client for Node.js." https://github.com/sidorares/node-mysql2

[7] [7] Redis Ltd., "Redis Documentation: Data Structures, Caching and Key Expiration." https://redis.io/docs/latest/

[8] [8] OpenJS Foundation, "Node.js Documentation: Crypto Module and PBKDF2." https://nodejs.org/api/crypto.html

[9] [9] Zod, "Zod Documentation: TypeScript-first Schema Validation." https://zod.dev/

GitHub Repository:
https://github.com/Praneeth-236/dbms_defence_asset_management

Appendix A. Verified Project File Inventory

- web-app/app/page.tsx — dashboard data loader and command-center layout.
- web-app/app/globals.css — shared theme, responsive layout, panels, tables, forms, and dashboard artwork.
- web-app/lib/cache.ts — Redis client, TTL, fallback, invalidation, and status support.
- web-app/lib/auth.ts and web-app/app/api/auth/login/route.ts — password verification, sessions, and safe diagnostics.
- web-app/scripts/provision-demo-user.mjs — confirmation-gated local provisioning.
- submission/00_schema_verification.sql through submission/06_read_only_evidence_checklist.sql — DBMS evidence scripts.

Appendix B. Database Table Catalogue

- COUNTRY stores country and region information. SERVICE belongs to a country. UNIT belongs to a service.
- ASSET stores the authoritative asset record and references UNIT and ASSET_TYPE.
- SUBSYSTEM and COMPONENT model the internal asset hierarchy.
- COMPONENT_DEPENDENCY, COMPONENT_SPARE, and SUPPLIER_SPARE model relationship facts.
- FAILURE_EVENT, INSPECTION, MAINTENANCE_WORK_ORDER, and MAINTENANCE_EVENT model operational history.
- SPARE_PART and SUPPLIER model supply readiness. USER supports authenticated application operations.

Appendix C. Key SQL Query Catalogue

- Asset count: SELECT COUNT(*) FROM ASSET.
- Open work orders: filter MAINTENANCE_WORK_ORDER by Open and InProgress.
- Low stock: compare SPARE_PART.StockQuantity with ReorderLevel.
- Failure severity: filter FAILURE_EVENT by Severity.
- Readiness status: group ASSET by Status.
- Category distribution: join ASSET to ASSET_TYPE and group by category.
- Dependency alerts: join component dependency, components, subsystem, and asset.

Appendix D. Application API Catalogue

- POST /api/auth/login — validates credentials and creates a session.
- POST /api/auth/logout — clears the current session.
- GET /api/health/redis — reports non-sensitive Redis status.
- Asset, work-order, dependency, and inventory API routes preserve existing contracts and role checks.

Appendix E. Manual Demonstration Script

- Start MySQL and Redis using the approved local environment.
- Set AUTH_SESSION_SECRET locally without committing it.
- Run the provisioning script only after confirming the target database and reviewing the proposed user.
- Start the web application and sign in with the locally provisioned account.
- Open the dashboard, then call /api/health/redis and inspect EXISTS dashboard:summary and TTL dashboard:summary.
- Create or update an authorized record, reload the dashboard, and confirm invalidation followed by repopulation.

Appendix E. Test Evidence Tables

- Complete the following table after manual execution.

Table A.1 : Test Evidence Tables

Test	Expected result	Observed result	Evidence
Typecheck	No TypeScript errors	Passed during implementation	[PLACEHOLDER: terminal screenshot]
Lint	No ESLint warnings/errors	Passed during implementation	[PLACEHOLDER: terminal screenshot]
Production build	Build completes	Passed during implementation	[PLACEHOLDER: terminal screenshot]
Schema verification	No integrity violations	[PLACEHOLDER: run manually]	[PLACEHOLDER]
Redis cache	Key present with positive TTL	[PLACEHOLDER: authenticated run]	[PLACEHOLDER]